



Console.log(` NodeJS Security Essentials `)

Essential Node.js Security
for Express Web Applications

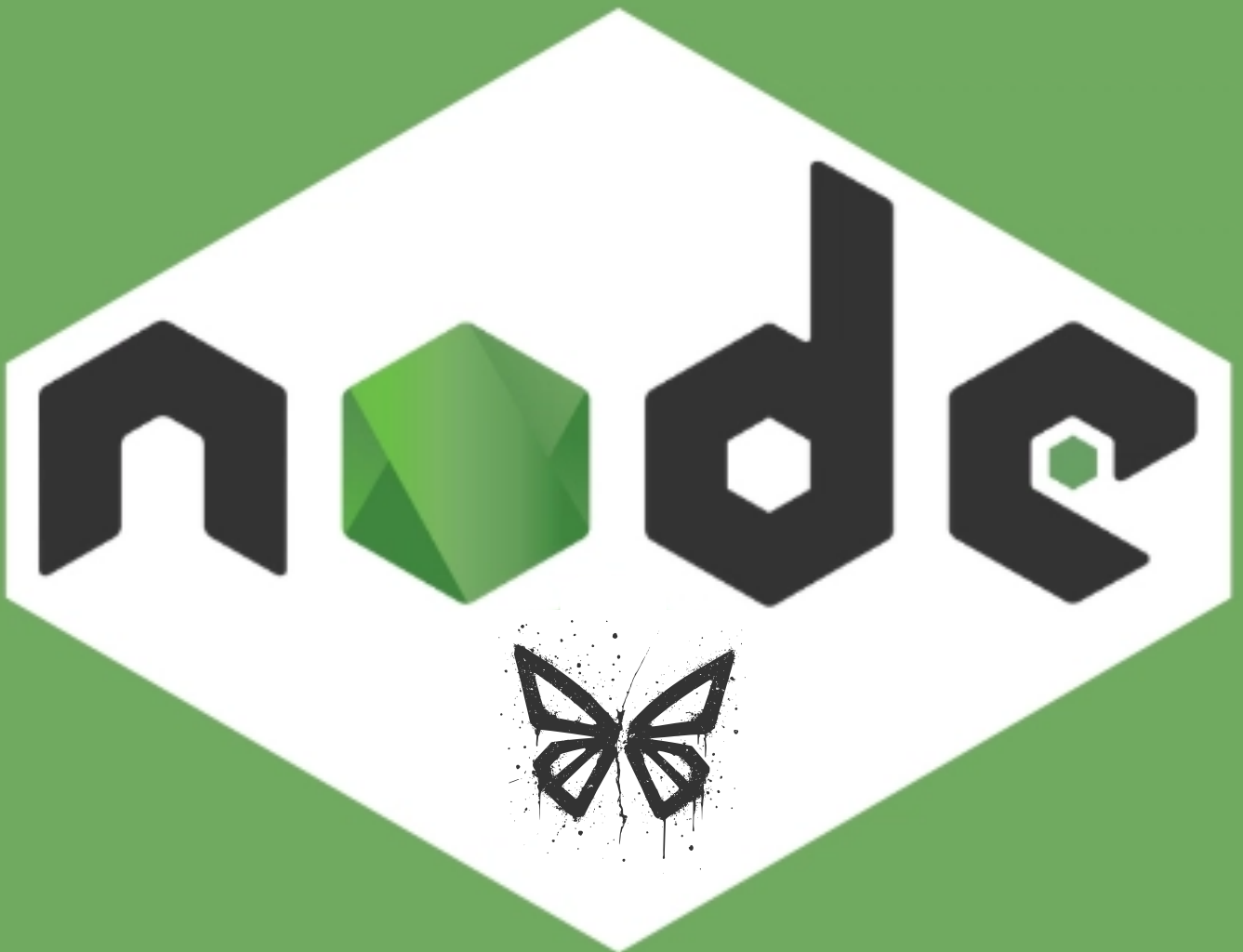




Table of Contents

Author Speech	v
Chapter one	i
HTTP Headers Security	i
Security Headers Caveats	i
Node.js Packages for HTTP Security Headers	ii
Helmet	ii
Strict Transport Security	iii
The Risk	iii
The Solution	iv
Helmet Implementation	iv
Scenario I	v
X-Frame-Options	v
The Risk	v
The Solution	vi
Helmet Implementation	vi
Scenario II	vi
Content-Security-Policy	vii
The Risk	vii





The Solution	10
Helmet Implementation	12
Gradual CSP Implementation	14
Scenario 11	10
Other HTTP headers	14
X-XSS-Protection	14
X-Content-Type-Options	10
Summary	10
Chapter two	14
Secure Session Management	14
Session Security Risks	10
Session Security in Node.js and ExpressJS	11
Secure your traffic with HTTPS	11
Secure cookie access to communication protocol only	12
Secure storage of session cookie	12
Obscuring the session identifier	14
Secure session ID	10
Re-generating session IDs	14





Reference for secure session configuration	PU
Summary	PU
Chapter three	PA
Hardening ExpressJS	PA
Security Through Obscurity	PA
Brute-Force Protection	PA
Advanced Functionality Limiting	PA
body-parser middleware	PA
Summary	PA
Chapter four	PA
Cross-Site Request Forgery (CSRF)	PA
The Risk	PA
The Solution	PA
CSRF Tokens	PA
CSRF Tokens Implementation	PA
Request Body	PA
CSRF Cookie	PA
CSRF Token Header	PA





CSRF Libraries – under the hood	۴۳
ExpressJS csrf Library	۴۴
csrf With Cookies	۴۴
csrf With Session	۴۶
ExpressJS With lusca Library.....	۴۷
lusca With Session	۴۸
Summary	۴۸
Cross-Site Scripting (XSS)	D۰
The Risk	D۱
Variations of XSS syntax	D۳
The Solution	D۴
Encoding libraries: node-esapi.....	D۶
Encoding libraries: xss-filters	D۷
Summary	D۹





Author Speech

سلام و درود

این کتابچه کوچک برای راهنمایی ابتدایی در **Node JS** برای وب اپلیکیشن ها شماست. سعی شده است مفاهیم ابتدایی به کمک نمونه کد ها توضیح داده شود تا در مسیر برنامه نویسی شما کننده باشد.

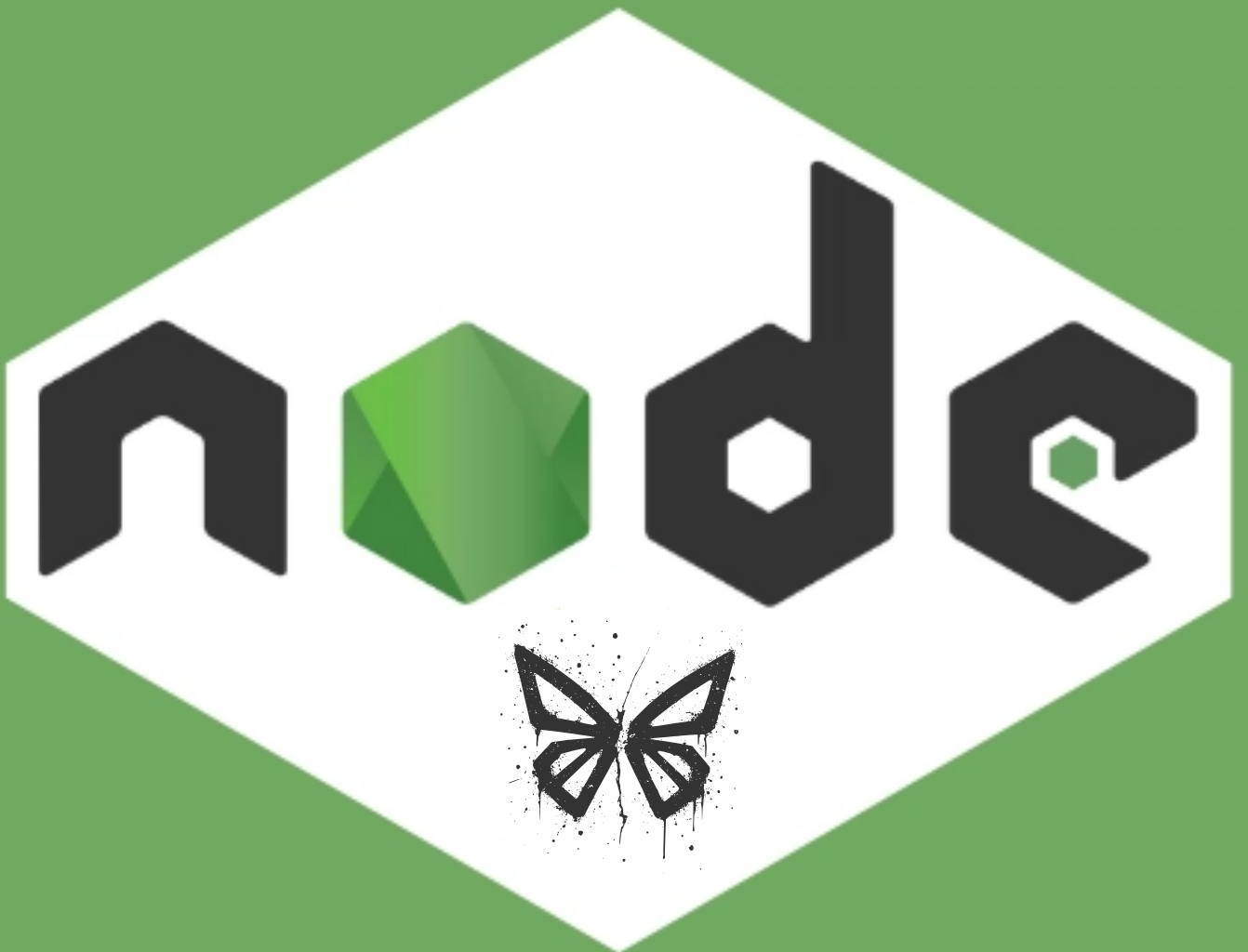
سرفصل ها از روی کتاب : **Essential Node.js Security Book by Liran Tal** امید است که این کتاب برای شما مفید واقع شود.

راه ارتباطی با ما : **t.me/Chaos_Nexus_Tech**





Console.log(` Chapter one `)





Chapter one

HTTP Headers Security

توسعه برنامه‌های وب به این معنی است که برنامه ما به پروتکل‌های ارتباطی وابسته است که از قبل مجموعه‌ای از استانداردها را برای نحوه انتقال داده‌ها و نحوه مدیریت آن به شیوه‌ای امن تعریف و پیاده‌سازی کرده‌اند. مرورگرها از هدرهای ارسال شده از طریق **HTTP** (عمدتاً اتصالات امن **HTTPS**) برای اجرا و تأیید چنین استانداردهای ارتباطی و همچنین سیاست‌های امنیتی استفاده می‌کنند. استفاده از این هدرهای **HTTP** برای افزایش امنیت کدی که **در سمت کلاینت مرورگر** اجرا می‌شود، روشی سریع و کارآمد برای کاهش آسیب‌پذیری‌های امنیتی و افزودن دفاع در عمق است.

Security Headers Caveats¹

استفاده از هدرهای امنیتی می‌تواند یک استراتژی عالی برای کمک به جلوگیری از آسیب‌پذیری‌های امنیتی باشد، اما یک اشتباه رایج این است که صرفاً برای کاهش چنین مسائلی به آنها تکیه کنیم. دلیل این امر این است که پاسخ به یک درخواست با یک هدر امنیتی به پشتیبانی، پیاده‌سازی و پایبندی مرورگر به مشخصات لازم برای اجرای آن بستگی دارد.

به همین ترتیب، هدرهای امنیتی باید به عنوان یک مکانیسم امنیتی دفاعی در عمق استفاده شوند که به افزودن یک کنترل امنیتی کمک می‌کند، اما نباید کنترل امنیتی واقعی برای دفاع در برابر آسیب‌پذیری‌هایی مانند **Cross-site Scripting** بین سایتی باشند.

¹ Caveat: پیش بینی احتیاطی





Node.js Packages for HTTP Security Headers

پاسخ به درخواست‌ها با هدرهای HTTP مرتبط با امنیت، یک نگرانی است و به همین دلیل، می‌تواند با هر چارچوب وب یا با ماژول‌های اصلی http یا https که Node.js ارائه می‌دهد، به کار گرفته شود. کتابخانه Helmet یک انتزاع و پیکربندی آسان برای هدرهای امنیتی HTTP مختلف که باید استفاده شوند، ارائه می‌دهد و انتخابی عالی برای برنامه‌های وب Node.js مبتنی بر Express است.

Helmet

Helmet یک میان‌افزار در سطح برنامه برای ExpressJS است که پشتیبانی گسترده‌ای از هدرهای امنیتی HTTP، مانند X-XSS-Protection برای کاهش آسیب‌پذیری‌های امنیتی مختلف اسکریپت‌نویسی بین‌سایتی (XSS) مرتبط با مرورگر، محافظت از گزینه‌های X-Frame و بسیاری دیگر ارائه می‌دهد. Helmet، که مجموعه‌ای از توابع میان‌افزار برای افزایش امنیت هدرهای HTTP است، به خوبی نگهداری و به‌روز نگه داشته می‌شود. به همین دلیل، انتخاب خوبی برای گنجاندن در برنامه‌های وب Node.js شما است.

Helmet با معرفی توابع میان‌افزار برای ExpressJS کار می‌کند که به درخواست‌های ورودی با مجموعه‌ای از هدرها پاسخ می‌دهند.

Quick start

```
import helmet from "helmet";

const app = express();

app.use(helmet());
```

^۱ Link : <https://github.com/helmetjs/helmet> , [helmetjs.github.io/](https://github.com/helmetjs/helmet)





Strict Transport Security

امنیت انتقال سخت گیرانه (**Strict Transport Security**) که با نام **HSTS** نیز شناخته می‌شود، یک استاندارد پروتکل برای اجرای اتصالات امن به سرور از طریق **HTTP** بر روی **SSL/TLS** است. **HSTS** با استفاده از هدر **HTTP** به نام **Strict-Transport-Security** پیکربندی و از سرور به هر کلاینت وب **HTTP** منتقل می‌شود. این هدر، یک بازه زمانی را مشخص می‌کند که در طی آن مرورگر فقط باید از طریق یک اتصال امن **HTTP (HTTPS)** ارتباط برقرار کند.

نکته: وقتی یک هدر **Strict-Transport-Security** از طریق یک اتصال **HTTP** ناامن

ارسال می‌شود، مرورگر وب آن را نادیده می‌گیرد زیرا اتصال از ابتدا ناامن بوده است.

در درخواست‌های بعدی پس از تنظیم هدر، مرورگر با یک سرویس پیش‌بارگذاری، مانند سرویس گوگل، مشورت می‌کند تا مشخص کند که آیا وب‌سایت مورد نظر **HSTS** را انتخاب کرده است یا خیر.

The Risk

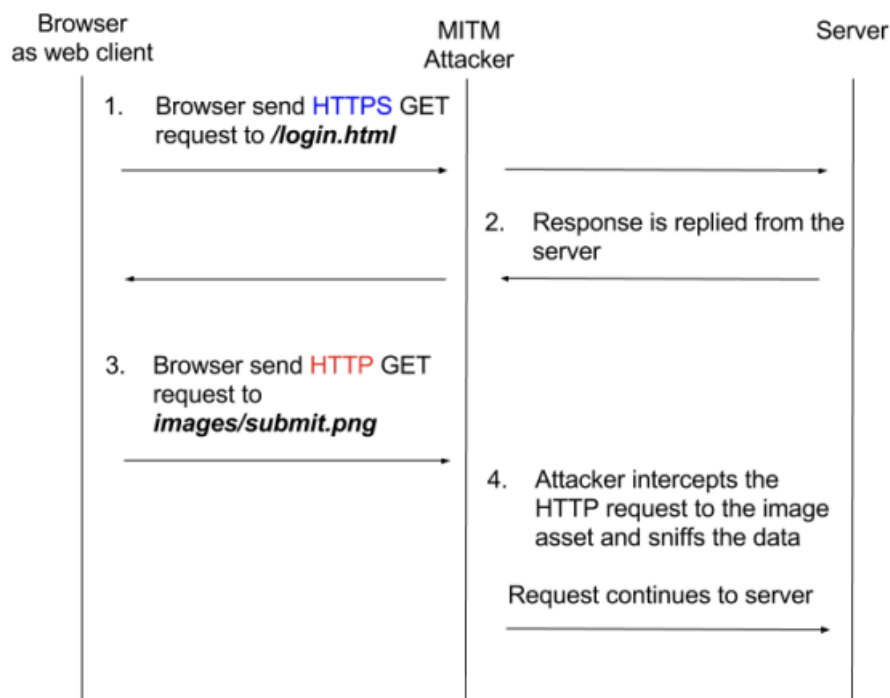
خطراتی که ممکن است هنگام برقراری ارتباط از طریق اتصال امن **HTTPS** ایجاد شود این است که یک کاربر مخرب می‌تواند حمله‌ی مرد میانی (**MITM**) را انجام دهد و درخواست‌های آینده به وب سرور را به استفاده از اتصال **HTTP** کاهش دهد، بنابراین می‌تواند تمام داده‌هایی را که از طریق آن جریان می‌یابد، شنود و بخواند.

ارسال درخواست‌های ناامن **HTTP** به وب سرور، حتی اگر در ابتدا یک اتصال **HTTPS** برقرار شده باشد، به خودی خود خود مشکلی ایجاد نمی‌کند، زیرا کاربر از دلیل وقوع این اتفاق بی‌اطلاع است و لزوماً به حمله **MITM** مشکوک نمی‌شود. شاید سرور دارای یک نقطه پایانی **REST** باشد که هنوز از **HTTPS** پشتیبانی نمی‌کند.





در نمودار جریان زیر، می‌توانیم یک سناریوی نمونه را ببینیم که در آن سرور یک فایل **HTML** برای صفحه ورود به مرورگر برمی‌گرداند، که شامل برخی منابع قابل دسترسی از طریق **HTTP** (<http://cdn.server.com/images/submit.png>)، مانند تصویر دکمه ارسال، است.



اگر یک مهاجم بتواند یک حمله **MITM** انجام دهد و به اصطلاح "روی سیم بنشیند" تا هرگونه ترافیک رمزگذاری نشده‌ای را که از آن عبور می‌کند، گوش دهد و بو بکشد (**listen and sniff**)، می‌تواند به درخواست‌های **HTTP** که ممکن است شامل داده‌های حساس باشند، دسترسی پیدا کرده و آنها را بخواند. حتی سناریوهای بدتر ممکن است شامل منابع **HTTP** تنظیم شده برای نقاط پایانی **POST** یا **PUT** باشد که در آنها داده‌های واقعی ارسال می‌شوند و می‌توانند **sniff** شوند.





The Solution

برای اینکه برنامه‌های وب بتوانند یک اتصال امن **HTTPS** را اجرا کنند، باید هدر **Strict-Transport-Security** را با یک مقدار مشخص ارسال کنند که نشان دهنده مدت زمانی بر حسب ثانیه است که کلاینت وب باید درخواست‌های آینده را از طریق یک اتصال امن **HTTPS** ارسال کند. مثلاً به مرورگر دستور دهد که تمام درخواست‌های ارسالی به سرور را برای ساعت آینده به **HTTPS** ارتقا دهد:

Strict-Transport-Security: max-age=3600

Helmet Implementation

برای استفاده از کتابخانه **HSTS** هلمت، باید بسته **npm** را دانلود کنیم و آن را به عنوان یک **dependency** بسته به پروژه **Node.js** که روی آن کار می‌کنیم، اضافه خواهیم کرد:

npm install helmet --save

بیاید میان‌افزار **hsts** را طوری تنظیم کنیم که به یک کلاینت وب مانند مرورگر نشان دهد که فقط باید درخواست‌های **HTTPS** را برای ماه آینده به نام میزبان سرور ما ارسال کند:

```
const helmet = require( "helmet" );
// Set the expiration time of HTTPS requests to the
// server to 1 month, specified in milliseconds
const reqDuration = 2629746000 ;
app.use(
  helmet.hsts({
    maxAge : reqDuration,
  })
);
```





در قطعه کد بالا، ما به برنامه **ExpressJS** دستور می‌دهیم که از میان‌افزار **hsts** استفاده کند و به همه درخواست‌ها با مجموعه هدر **Strict-Transport-Security** پاسخ دهد.

توجه داشته باشید که اگر به هر دلیلی مرورگر چندین دستورالعمل هدر **HSTS** دریافت کند، فقط بر اساس اولین مورد ارسالی، سیاست را رعایت و اجرا می‌کند.

معمول است که سرورهای وب زیر دامنه‌هایی برای دریافت دارایی‌ها یا برقراری تماس‌های **REST API** داشته باشند، که در این صورت ما نیز می‌خواهیم از آنها محافظت کنیم و درخواست‌های **HTTPS** را اجرا کنیم. برای انجام این کار، می‌توانیم پارامتر اختیاری زیر را به شیء گزینه‌های **hsts** اضافه کنیم:

includeSubDomains : true ;

نکته : اگر لازم باشد به مرورگر دستور غیرفعال کردن «**Strict-Transport-Security**» داده شود، یک سرور می‌تواند با تنظیم حداکثر سن این هدر روی صفر پاسخ دهد که منجر به این می‌شود که مرورگر فوراً سیاست را منقضی کرده و دسترسی از طریق یک اتصال **HTTP** ناامن را فعال کند.

Scenario I

برویم سناریو شماره یک را با یک دیگر حل کنیم !





X-Frame-Options

هدر **HTTP** مربوط به **X-Frame-Options** برای کاهش حمله‌ای به نام **Clickjacking** معرفی شد. این هدر به مهاجم اجازه می‌دهد تا عناصر صفحه مانند دکمه‌ها و ورودی‌های متنی را با پنهان کردن نمای آنها در پشت صفحات وب واقعی که با استفاده از یک عنصر **HTMLiframe** یا اشیاء مشابه روی صفحه نمایش داده می‌شوند، پنهان کند.

((اطلاعیه منسوخ شدن))

هدر **X-Frame-Options** هرگز به عنوان بخشی از مشخصات رسمی استاندارد نشد، اما بسیاری از مرورگرهای محبوب امروزی هنوز از آن پشتیبانی می‌کنند. جانشین آن، هدر **Content-Security-Policy (CSP)** است که در بخش بعدی به آن پرداخته خواهد شد و باید بر پیاده‌سازی **CSP** برای برنامه‌های وب تازه ساخته شده تمرکز کرد.

The Risk

حمله **Clickjacking**، که با نام اصلاح **UIredressing** نیز شناخته می‌شود، در مورد گمراه کردن کاربر برای انجام عملیاتی به ظاهر ساده و بی‌ضرر است، در حالی که در واقع کاربر در حال کلیک کردن روی دکمه‌هایی است که متعلق به عناصر دیگر هستند، یا تایپ متن در یک فیلد ورودی که تحت کنترل مهاجم است.

نمونه‌های رایج استفاده از حمله کلیک‌ربایی:

۱. اگر یک وب‌سایت حساب بانکی یا ایمیل از هدر **HTTP** با عنوان **X-Frame-Options** استفاده نکند، یک مهاجم مخرب می‌تواند آنها را در یک **iframe** رندر کند و فیلم‌های ورودی خود را در محل دقیق ورودی وب‌سایت بانکی یا ایمیل برای نام کاربری و رمز عبور و ثبت اطلاعات اعتبار شما قرار دهد.

۲. یک برنامه وب برای چت ویدیویی یا صوتی که ناامن است، می‌تواند توسط این حمله مورد سوء استفاده قرار گیرد تا کاربر به اشتباه فرض کند که فقط روی صفحه کلیک می‌کند یا بازی می‌کند، در حالی که در واقع این سری کلیک‌ها در واقع وب‌کم شما را روشن می‌کند.





The Solution

برای کاهش این مشکل، یک وب سرور می‌تواند به درخواست مرورگر با یک هدر **HTTP** از نوع **X-Frame-Options** که روی یکی از مقادیر ممکن زیر تنظیم شده است، پاسخ دهد:

۱. **DENY** - مشخص می‌کند که وبسایت نمی‌تواند در یک عنصر **HTML** از نوع **frame**، **iframe** یا **object** رندر شود.

۲. **SAMEORIGIN** - مشخص می‌کند که وبسایت فقط در صورتی می‌تواند رندر شود که در یک عنصر **HTML** از نوع **frame**، **iframe** یا **object** از همان دامنه‌ای که درخواست از آن سرچشمه گرفته است، جاسازی شده باشد.

۳. **ALLOW-FROM <URI>** - مشخص می‌کند که وبسایت می‌تواند از طریق **URI** ارائه شده، فریم‌بندی و رندر شود. توجه به این نکته مهم است که نمی‌توانید چندین مقدار **URI** را مشخص کنید، اما فقط به یک مقدار محدود هستید.

چند مثال برای نشان دادن نحوه تنظیم این هدر عبارتند از:

X-Frame-Options: ALLOW-FROM http://www.mydomain.com

X-Frame-Options: DENY

مراقب پروکسی‌ها باشید

پروکسی‌های وب (**Web proxies**) اغلب به عنوان وسیله‌ای برای ذخیره‌سازی موقت استفاده می‌شوند و به طور طبیعی دستکاری زیادی در هدرها انجام می‌دهند. مراقب پروکسی‌هایی باشید که ممکن است این یا سایر هدرهای مرتبط با امنیت را از پاسخ حذف کنند.





Helmet Implementation

با **helmet**، پیاده‌سازی این هدر به سادگی درخواست پکیج **helmet** و استفاده از شیء **app** در **ExpressJS** برای دستور دادن به **ExpressJS** جهت استفاده از میان‌افزار **xframe** ارائه شده توسط **helmet** است. برای تنظیم **X-Frame-Options** برای رد کامل تمام اشکال جاسازی:

```
const helmet = require( "helmet" );

app.use(
  helmet.frameguard({
    action : "deny" ,
  })
);
```

به طور مشابه، می‌توانیم با ارائه شیء گزینه‌های زیر، اجازه دهیم فریم‌ها فقط از یک مبدا یکسان ارسال شوند:

```
{action: "sameorigin";}
```

یا برای اینکه اجازه دهیم فریم‌ها از یک میزبان مشخص ارسال شوند:

```
{ action : 'allow-from' , domain : 'https://mydomain.com' }
```

Scenario ۲

برویم سناریو شماره دو را با یک دیگر حل کنیم !





Content-Security-Policy

همانطور که قبلاً در مورد هدر **X-Frame-Options** بررسی شد، حملات زیادی مربوط به تزریق محتوا در مرورگر کاربر وجود دارد - چه حمله **Clickjacking** باشد و چه انواع دیگری از حملات مانند **Cross-Site-Scripting (XSS)**. یکی دیگر از بهبودهای مجموعه هدرهای قبلی که تاکنون بررسی کردیم، هدری است که می‌تواند به مرورگر بگوید به کدام محتوا اعتماد کند.

این به مرورگر اجازه می‌دهد تا از تلاش‌های تزریق محتوا که در سیاست تعریف شده توسط صاحب برنامه مورد اعتماد نیستند، جلوگیری کند. با یک سیاست امنیتی محتوا (**CSP**) می‌توان از طیف وسیعی از حملات، از جمله **Cross-site scripting** و سایر تزریق‌های محتوا، جلوگیری کرد. پیاده‌سازی **CSP** استفاده از هدر **X-Frame-Options** را منسوخ می‌کند.

The Risk

استفاده از هدر سیاست امنیت محتوا از چندین مشکل مانند موارد زیر جلوگیری و آنها را کاهش می‌دهد:

- کد جاوا اسکریپت درون خطی مشخص شده با برچسب‌های **<script>** و هرگونه رویداد **DOM** که باعث اجرای جاوا اسکریپت می‌شود مانند **onClick()** و غیره.
- کد **CSS** درون خطی مشخص شده از طریق برچسب **<style>** یا عناصر ویژگی.

The Solution

با لیست‌های مجاز **CSP**، می‌توانیم پیکربندی‌های زیادی را برای محتوای قابل اعتماد مجاز کنیم و به همین ترتیب، راه‌اندازی اولیه می‌تواند به مجموعه‌ای از دستورالعمل‌های پیچیده تبدیل شود. بیا یک دستورالعمل به نام **connect-src** را بررسی کنیم. این دستورالعمل برای کنترل اینکه مرورگر مجاز به اتصال به کدام ریموت‌ها از طریق **XHR** یا **WebSockets** است، استفاده می‌شود. مقادیر قابل قبولی که می‌توانیم برای این دستورالعمل تنظیم کنیم:

- **'none'** - عدم اجازه به فراخوانی‌های ریموت مانند **XHR** به هیچ وجه
- **'self'** - فقط اجازه فراخوانی‌های ریموت به دامنه خودمان را می‌دهیم (نام دقیق دامنه/میزبان. زیر دامنه‌ها مجاز نیستند)





مثالی برای این دستورالعمل که توسط وب سرور تنظیم شده و فقط امکان فراخوانی‌های از راه دور به دامنه خودمان و دامنه API گوگل را فراهم می‌کند:

Content-Security-Policy: connect-src 'self' https://apis.google.com;

دستورالعمل دیگری برای کنترل لیست مجاز منابع جاوا اسکریپت، **script-src** نامیده می‌شود. این دستورالعمل با اطلاع دادن به مرورگر در مورد اینکه هنگام ارزیابی و اجرای کد منبع جاوا اسکریپت به کدام منابع محتوا اعتماد کند، به کاهش حملات اسکریپت‌نویسی بین سایتی (XSS) کمک می‌کند.

script-src از کلمات کلیدی **"none"** و **"self"** به عنوان مقادیر پشتیبانی می‌کند و شامل گزینه‌های زیر است:

- **'unsafe-inline'** - به هر کد منبع جاوا اسکریپت درون خطی مانند **<script>** و رویدادهای DOM که مانند **onClick()** یا **javascript:URI** ها را فعال می‌کنند، اجازه می‌دهد. همچنین بر **CSS** برای تگ‌های درون خطی تأثیر می‌گذارد.

- **'unsafe-eval'** - اجازه اجرای کد با استفاده از **eval()** را می‌دهد.

برای مثال، سیاستی برای اجازه اجرای جاوا اسکریپت فقط از دامنه خودمان و از دامنه گوگل، و همچنین اجازه اجرای کد جاوا اسکریپت درون خطی:

Content-Security-Policy: script-src 'self' https://apis.google.com 'unsafe-inline'

لیست کاملی از دستورالعمل‌های پشتیبانی‌شده را می‌توانید در صفحه دستورالعمل‌های سیاست **CSP** در **MDN** بیابید، اما بیایید به برخی از گزینه‌های رایج دیگر و مقادیر آنها بپردازیم.

- **default-src** - در جایی که یک دستورالعمل مقداری ندارد، به طور پیش‌فرض روی یک پیکربندی باز و بدون محدودیت تنظیم می‌شود. تنظیم پیش‌فرض برای همه گزینه‌های پیکربندی نشده ایمن‌تر است و این هدف دستورالعمل **default-src** است.





• **script-src** - دستورالعملی برای تعیین مکان‌هایی که اجازه بارگذاری یا اجرای منابع جاوا اسکریپت را از آنها می‌دهیم. اگر روی مقدار «**self**» تنظیم شود، هیچ تگ جاوا اسکریپت درون خطی، مانند **<script>**، مجاز نیست و فقط منابع از دامنه خودمان مجاز هستند.

همچنین باید توجه داشت که پیکربندی **CSP** باید با پیاده‌سازی معماری برنامه وب شما مطابقت داشته باشد. اگر بلوک‌های **<script>** درون خطی را رد کنید، تیم تحقیق و توسعه شما باید از این موضوع آگاه و برای آن آماده باشد، زیرا در غیر این صورت، این امر باعث اختلال در ویژگی‌ها و عملکردهای موجود در کدی خواهد شد که به بلوک‌های کد جاوا اسکریپت درون خطی وابسته است.

Helmet Implementation

با استفاده از **helmet** می‌توانیم یک سیاست امن برای محتوای قابل اعتماد پیکربندی کنیم.

با توجه به پتانسیل پیکربندی پیچیده، چندین سیاست مختلف را در بلوک‌های کد کوچک‌تر بررسی خواهیم کرد تا به راحتی توضیح دهیم که هنگام پیاده‌سازی **CSP** چه اتفاقی می‌افتد.

کد **Node.js** زیر، میان‌افزار **CSP** مربوط به **helmet** را در هر درخواست اضافه می‌کند تا سرور با یک هدر **CSP** و یک سیاست امنیتی ساده پاسخ دهد.

ما یک لیست مجاز تعریف می‌کنیم که در آن کد جاوا اسکریپت و منابع **CSS** فقط مجاز به بارگیری از مبدا فعلی هستند که دقیقاً نام میزبان یا دامنه است (هیچ زیر دامنه‌ای مجاز نخواهد بود):

```
const helmet = require( "helmet" );
app.use(
  helmet.contentSecurityPolicy({
    directives : {
      scriptSrc : [ "'self'" ],
      styleSrc : [ "'self'" ],
    },
  })
);
```





مهم است به یاد داشته باشید که اگر هیچ سیاست پیش فرضی مشخص نشده باشد، تمام انواع دیگر سیاست‌های محتوا باز و مجاز هستند، و همچنین برخی از سیاست‌های محتوا به سادگی پیش فرض ندارند و باید برای لغو شدن مشخص شوند.

بیایید سیاست محتوای زیر را برای برنامه وب خود بسازیم:

- به طور پیش فرض، اجازه دهید منابع فقط از مبدا دامنه خودمان یا از **CDN** آمازون ما بارگیری شوند.
- منابع جاوا اسکریپت به دامنه خودمان و دامنه کتابخانه‌های میزبانی شده گوگل محدود شده‌اند، بنابراین می‌توانیم **AngularJS** را از گوگل بارگیری کنیم.
- از آنجا که برنامه وب ما به هیچ نوع جاسازی **iframe** نیاز ندارد، چنین اشیاء را غیرفعال می‌کنیم (به **objectSrc** اشاره دارد).
- فرم‌ها فقط باید به مبدا دامنه خودمان ارسال شوند.



```
var helmet = require( "helmet" );
app.use(
  helmet.contentSecurityPolicy({
    directives : {
      defaultSrc : [ "'self'" , "https://cdn.amazon.com" ],
      scriptSrc : [ "'self'" , "https://ajax.googleapis.com" \
    ],
    childSrc : [ "'none'" ],
    objectSrc : [ "'none'" ],
    formAction : [ "'none'" ],
  },
})
);
```





Gradual[™] CSP Implementation

سیاست امنیت محتوای شما با رشد برنامه وب شما نیز رشد و تغییر خواهد کرد. با وجود دستورالعمل‌های متنوع، معرفی یک سیاست به طور همزمان می‌تواند چالش برانگیز باشد، بنابراین به جای اجرای سریع، سعی کنید رویکردی تدریجی داشته باشید.

هدر **CSP** دارای یک دستورالعمل داخلی است که به درک نحوه استفاده برنامه وب شما از سیاست محتوا کمک می‌کند. این دستورالعمل برای ردیابی و گزارش هرگونه اقدام انجام شده توسط مرورگر که سیاست امنیت محتوا را نقض می‌کند، استفاده می‌شود.

افزودن آن به هر برنامه وب در حال اجرا ساده است:

Content-Security-Policy : default-src 'self' ; report-uri https :// mydomain.com /report

پس از افزودن، مرورگر یک درخواست **POST** به **URI** ارسال می‌کند که در بدنه آن فرمت **JSON** برای هر چیزی که سیاست امنیت محتوا مبنی بر ارائه محتوا از مبدا خودمان را نقض می‌کند، ارائه شده است.

با میان‌افزار **csp** در **Helmet**، این مورد به راحتی پی‌گیری می‌شود:

```
const helmet = require( "helmet" );
app.use(
  helmet.csp({
    directives : {
      defaultSrc : [ "self" ],
      reportUri : "https://mydomain.com/report" ,
    },
  })
);
```





یکی دیگر از پیکربندی‌های مفید برای **helmet** زمانی که هنوز در حال ارزیابی یک سیاست امنیتی محتوا هستیم، این است که به مرورگر دستور دهیم فقط موارد نقض سیاست محتوا را گزارش دهد و آنها را مسدود نکند:

```
const helmet = require( "helmet" );
app.use(
  helmet.csp({
    directives : {
      defaultSrc : [ "self" ],
      reportUri : "https://mydomain.com/report" ,
    },
    reportOnly : true ,
  })
);
```

Scenario ۳

برویم سناریو شماره سه را با یک دیگر حل کنیم!





Other HTTP headers

هدرهای HTTP غیر استاندارد دیگری نیز وجود دارند و جزئی از هیچ یک از مشخصات رسمی مانند IANA نیستند، اما ارزش بررسی دارند زیرا لایه امنیتی دیگری را برای کاربران شما فراهم می کنند.

X-XSS-Protection

هدر **X-XSS-Protection** توسط IE۹ و IE۱۱ استفاده می شود و امکان فعال یا غیرفعال کردن قابلیت فیلتر اسکریپت نویسی بین سایتی (XSS) را که در مرورگر تعبیه شده است، فراهم می کند. فعال کردن فیلترینگ XSS برای هر مرورگر IE۹ و IE۱۱ که رندر می کند برنامه وب شما نیاز به ارسال هدر HTTP زیر دارد:

X-XSS-Protection : 1 ; mode=block

با استفاده از **Helmet**، این محافظت را می توان با استفاده از قطعه کد زیر فعال کرد:

```
const helmet = require("helmet");  
app.use(helmet.xssFilter());
```





X-Content-Type-Options

هدف از این هدر، آموزش مرورگر برای جلوگیری از حدس زدن نوع محتوای وب سرور است که ممکن است منجر به رندر نادرستی نسبت به آنچه وب سرور در نظر داشته است، شود. هدر **HTTP** مربوط به **X-Content-Type-Options** توسط **IE**، **Chrome** و **Opera** استفاده می‌شود و برای کاهش حمله مبتنی بر **MIME** استفاده می‌شود.

مثالی از تنظیم این هدر:

X-Content-Type-Options: nosniff

در **helmet** :

```
const helmet = require("helmet");  
app.use(helmet.noSniff());
```

Summary

در این فصل، با پیاده‌سازی هدرهای **HTTP** برای افزایش امنیت، کنترل‌های امنیتی مرورگر را معرفی کردیم. ما در مورد **Helmet** به عنوان کتابخانه‌ای که می‌توان به راحتی به هر پروژه **ExpressJS** اضافه کرد، آموختیم و آن را برای ارائه امنیت بیشتر پیکربندی کردیم. هدرهای امنیتی **HTTP** که بررسی کردیم عبارتند از:

• **Strict Transport Security**

• **X Frame Options**

• **Content Security Policy**

• **X XSS Protection**

• **X Content Type Options**

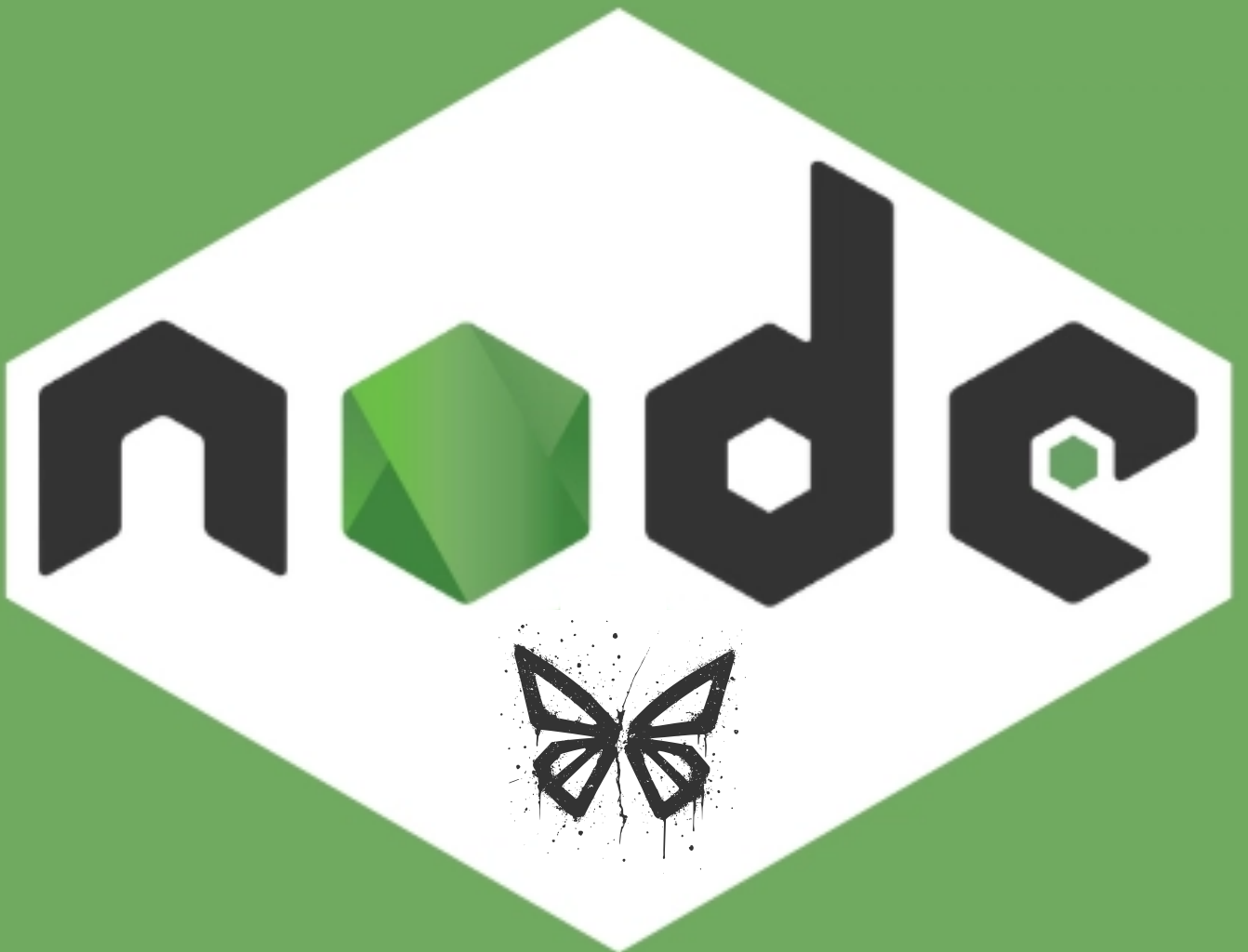




Console.log(`

Chapter two

`)





Chapter two

Secure Session Management

اگر برنامه وب شما **stateless**^۴ است و نیازی به سفارشی سازی کاربر و ردیابی کاربر ندارد، احتمالاً حتی لازم نیست نگران مدیریت کاربران و جلسات باشید.

سناریوی دیگر این است که شما نیاز دارید محتوای سفارشی شده برای کاربران ارائه دهید، به آنها اجازه دهید وارد سیستم شوند و برخی اقدامات را انجام دهند، یا یک فعالیت مرتبط با کاربر را حفظ کنید. اینجاست که مسائل پیچیده می شوند و نیاز به توجه دقیق به جزئیات بسیار بیشتری از امنیت اطلاعات دارند.

HTTP یک پروتکل ارتباطی بدون وضعیت (**stateless**) است، بنابراین نیاز به مکانیزمی برای ردیابی و نگهداری اقدامات کاربر هنگام تعامل با برنامه های وب - جلسات - را ایجاد می کند. تمرکز این فصل بر مدیریت جلسه مبتنی بر کوکی ها خواهد بود که رایج ترین روش نگهداری جلسات برای برنامه های وب است.

^۴ A stateless application is one that needs no knowledge of previous interactions and stores no session information

یک برنامه **stateless**، برنامه ای است که نیازی به دانش تعاملات قبلی ندارد و هیچ اطلاعاتی از جلسه را ذخیره نمی کند.





Session Security Risks

مدیریت نادرست نشست در برنامه‌های وب ممکن است منجر به آسیب‌پذیری‌های متعددی شود که می‌توانند توسط مهاجمان مورد سوءاستفاده قرار گیرند.

برای درک عمیق امنیت جلسه، ابتدا باید درک اولیه‌ای از مدیریت جلسات داشته باشیم. یک وب سرور با ذخیره یک توکن در مرورگر کاربر، که اغلب به عنوان شناسه جلسه شناخته می‌شود، تعامل مرور کاربر را ردیابی می‌کند و از آن برای شناسایی این کاربر منحصر به فرد برای درخواست‌ها و تعاملات بیشتر بین کاربر و وب سرور استفاده می‌کند.

بررسی چند نمونه از حملات مرتبط با نشست:

- **Session Fixation** - تثبیت نشست ؛ با استفاده از چندین بردار حمله، سعی می‌کند یک نشست معتبر در مرورگر به دست آورد و سپس نشست مرور قربانی را طوری تنظیم می‌کند که از نشست موجود متعلق به مهاجم استفاده کند. بردارهای حمله احتمالی عبارتند از اسکریپت‌نویسی بین‌سایتی (XSS)، تزریق متا تگ، پذیرش نشست و موارد دیگر.
- **Session Hijacking** - ربودن نشست ؛ یک مهاجم از بردارهای حمله مشابهی مانند XSS استفاده می‌کند و همچنین ممکن است از یک حمله MITM برای آشکار کردن شناسه نشست معتبر کاربر استفاده کند تا بتواند توسط مهاجم ربوده شده و از آن استفاده کند.

تاثیر این حمله آن است که مهاجم در یک جلسه بتواند همان دسترسی و امتیازاتی را داشته باشد که کاربر آن جلسه دارا می‌باشد. در مقایسه با حملات یونیکس که ممکن است از حملات افزایش امتیاز ریشه سوءاستفاده کنند، این احتمالاً بزرگترین خطر را برای برنامه‌های وب نیز ایجاد می‌کند.





نکته : در مورد مدیریت نشست، **OWASP** یک چک لیست مدیریت نشست بهروز شده را برای تأیید انطباق امنیت وب شما با استانداردهای امنیتی ارائه می‌دهد.

Session Security in Node.js and ExpressJS

ExpressJS از میان‌افزار **express-session** برای مدیریت نشست‌ها استفاده می‌کند. این پروژه به خوبی نگهداری، آزمایش شده و یک راه‌حل بالفعل برای مدیریت نشست‌ها در **Node.js** است.

نصب **express-session**:

```
npm install express-session --save
```

بخش‌های بعدی این فصل، نحوه پیکربندی ایمن یک سیاست مدیریت نشست امن را بررسی می‌کنند و گام به گام گزینه‌های موجود **express-session** را توسعه می‌دهند.

خلاصه این فصل، پیکربندی کامل مدیریت نشست را برای راحتی شما ارائه می‌دهد.

Secure your traffic with HTTPs

مسیریابی تمام ترافیک **HTTP** شما از طریق یک لایه سوکت امن مانند **SSL** یا **TLS**، مانع از شنود داده‌های شما توسط مهاجمان می‌شود و انجام حملات **MITM** برای استراق سمع ترافیک شما را برای آنها دشوارتر می‌کند.

کوکی‌ها ممکن است با یک پرچم در مرورگر کاربر تنظیم شوند تا به مرورگر دستور دهند که فقط هنگام کار با ارتباط **HTTPS**، کوکی‌ها را ارسال کند.

```
var session = require( 'express-session' );

app.use(session({
  cookie : {
    secure : true
  }
}));
```





Secure cookie access to communication protocol only

مرورگرها روشی برای دسترسی به اطلاعات کوکی سمت کلاینت از طریق استفاده رایج از شیء `document.cookie` ارائه می‌دهند.

هنگامی که یک برنامه وب در برابر حملات اسکریپت‌نویسی بین‌سایتی (XSS) آسیب‌پذیر می‌شود، می‌توان از آن برای اجرای هرگونه کد جاوا اسکریپت دلخواه سوءاستفاده کرد. برخی از نمونه‌ها عبارتند از دسترسی به اطلاعات کوکی و انجام هرگونه عملی روی آن، مانند ارسال آن به یک سرویس از راه دور که توسط مهاجم کنترل یا نظارت می‌شود، یا چاپ آن در خروجی صفحه HTML تولید شده.

برای کاهش این مشکل، می‌توانیم دسترسی به اطلاعات کوکی را محدود کنیم تا مرورگر بداند که فقط باید داده‌های کوکی را از طریق پروتکل ارتباطی HTTP/HTTPS از طریق سیم ارسال/دریافت کند. در این صورت هرگونه تلاش برای اجرای توابع جاوا اسکریپت برای دریافت شیء کوکی با شکست مواجه خواهد شد.

```
var session = require( 'express-session' );

app.use(session({
  cookie : {
    httpOnly : true
  }
}));
```





Secure storage of session cookie

کوکی جلسه حداقل برای شناسایی جلسه مرور کاربر استفاده می‌شود و در موارد دیگر ممکن است حاوی اطلاعات حساس‌تری در مورد کاربر و برنامه وب باشد. به دلیل داده‌های حساسی که کوکی در خود نگه می‌دارد، ماندگاری و ذخیره‌سازی آن در سمت کلاینت مرورگر، آن را به هدفی جذاب برای مهاجمان تبدیل می‌کند و بنابراین یکی دیگر از جنبه‌های بسیار مهم امنیتی است.

ما دو نوع ذخیره‌سازی کوکی را شناسایی می‌کنیم: کوکی‌های پایدار و کوکی‌های غیرپایدار. کوکی‌های پایدار با ویژگی و مقدار **Max-Age** یا **Expires** مشخص می‌شوند که میزان زمان ذخیره در فضای ذخیره‌سازی دیسک مرورگر را تعریف می‌کنند. کوکی‌های غیرپایدار که ایمن‌تر هستند، برای مدت زمان باقی‌مانده زمانی که فرآیند مرورگر باز است، در حافظه مرورگر ذخیره می‌شوند. پس از بسته شدن، هرگونه اطلاعات کوکی که در مرورگر ذخیره شده بود، دیگر در دسترس نیست.

با استفاده از **express-session**، مقدار **maxAge** به طور پیش‌فرض روی **null** تنظیم شده است که باعث ایجاد پیکربندی کوکی امن می‌شود، اما ارزش دارد که این مقدار را برای وضوح و به‌روزرسانی‌های آینده کتابخانه اجباری کنید:

```
var session = require( 'express-session' );
app.use(session({
  cookie : {
    maxAge : null ;
  }
}));
```





نکته: آیا تا به حال گزینه «مرا به خاطر بسپار» را فعال کرده‌اید؟

برنامه‌های وب از ذخیره دائمی داده‌های کوکی در حافظه دیسک کاربر استفاده می‌کنند تا تجربه کاربری راحت‌تری را فراهم کنند که نیازی به ورود همیشگی کاربر نداشته باشد. اگر کوکی روی دیسک موجود باشد، جلسه کاربر فعال می‌ماند و می‌تواند از نقطه‌ای که قطع شده ادامه یابد.

Obscuring^۵ the session identifier

نام کوکی به نظر یک اطلاعات اولیه و بی‌اهمیت می‌رسد، زیرا صرفاً نام کوکی است، اما واقعیت به هیچ وجه ابتدایی نیست. انگشتنگاری حوزه‌ای در امنیت است که تلاش می‌کند سرویس‌ها و نسخه‌های آنها را که یک سرویس را پشتیبانی می‌کنند، بر اساس نحوه کار و آنچه ارسال می‌کنند، شناسایی کند. به عنوان مثال، یک برنامه وب رایج **PHP** نام کوکی را **PHPSESSION** قرار می‌دهد و به مهاجم این امکان را می‌دهد که از قبل بداند کدام پلتفرم یک برنامه وب را پشتیبانی می‌کند و چگونه و کجا باید مسیر حمله را متمرکز کند. در چنین مواردی، مهاجم از قبل اطلاعاتی را در مورد سیستم جمع‌آوری کرده است، بدون اینکه نیازی به انجام کاری داشته باشد.

در مورد **Node.js**، میان‌افزار **session** مربوط به **ExpressJS** به طور پیش‌فرض یک کوکی با نام **connect.sid** است. برای پنهان کردن این اطلاعات از دنیای خارج، می‌توانیم نام کوکی را به هر چیز دیگری تغییر دهیم:

```
var session = require( 'express-session' );

app.use(session({
  name : 'CR7' ;
}));
```

^۵ مبهم سازی ، پنهان سازی





نکته : اگر روزی خواستید کلاه‌سیاه به سر بگذارید و سیستم‌های دیگر را بررسی کنید، شاید بهتر باشد از «پایگاه داده کوکی» متعلق به **DWASP** استفاده کنید که اساساً فهرستی از نام کوکی‌های مورد استفاده فروشندگان است و شما را از دردسر انگشت‌نگاری این اطلاعات خلاص می‌کند.

Secure session ID

وقتی شناسه‌های جلسه کلید شناسایی کاربران هستند، تولید و تصادفی بودن آنها از اهمیت حیاتی برخوردار است. اگر آنها به روشی تولید شوند که امکان پیش‌بینی مقادیر آینده را فراهم کند، خطر بزرگی ایجاد می‌کنند زیرا یک مهاجم می‌تواند بر اساس تولید یک مقدار قابل پیش‌بینی تا زمانی که یک نتیجه مطابقت داشته باشد، سعی در حمله **brute force** به جلسات کاربر کند. دو جنبه برای شناسه‌های جلسه وجود دارد: تولید آنها و امضای آنها. با **express-session** ما به تأثیرگذاری بر هر دوی آنها دسترسی پیدا می‌کنیم و می‌توانیم شناسه‌های جلسه خود را بیشتر ایمن کنیم. امضای شناسه جلسه با یک رشته طولانی، آنتروپی بیشتری را فراهم می‌کند. یک مقدار خوب حداقل ۶۴ بیت خواهد بود که به هش **SHA-PD۶** شناسه کوکی تولید شده اضافه می‌شود:

```
var session = require( 'session' );
app.use(session({
  secret : 'THEWALL'
}));
```

احتمال تغییر شناسه‌های جلسه از حالت پیش‌فرض مورد استفاده توسط **express-session** که کتابخانه **uid-safe** است، کمتر است. با این حال، اگر نیاز به بازنویسی آن با مولد منحصر به فرد خود دارید، می‌توانید این کار را با ارائه یک تابع فراخوانی به ویژگی **genid** انجام دهید.





```
var session = require( 'session' );
app.use(session({
  genid : function (req) {
    return uniqueValue();
  }
}));
```

Re-generating session IDs

ماهیت حساس شناسه جلسه، روش‌های بیشتری را برای محافظت از آن می‌طلبد. یک دفاع خوب در برابر تثبیت جلسه و همچنین در برابر ربودن جلسه، تولید مجدد شناسه جلسه است. انجام این کار برای هر درخواست ممکن است زیاده‌روی باشد، اما انجام آن قبل از اقدامات حساسی که توسط کاربر انجام می‌شود و قبل از هرگونه افزایش امتیاز، کاملاً رایج است.

یک استراتژی خوب برای تولید مجدد شناسه جلسه در همه موارد زیر است:

- ورود کاربر

– پس از ورود کاربر به سیستم، باید یک شناسه جلسه جدید ایجاد شود.

- اقدامات حساس، بسته به برنامه، اما در اینجا یک مرجع وجود دارد:

– تغییر رمز عبور، به‌روزرسانی ایمیل و سایر اطلاعات شخصی که کاربر را شناسایی می‌کند

– اختصاص نقش‌ها و مجوزها به سایر کاربران

– حذف رکوردها

– انتقال پول در برنامه‌های بانکی یا خرید سهام و سناریوهای مشابه.





هنگام استفاده از **express-session**، میان افزار شیء **req.session** را با چندین متد و شیء پر می کند که دسترسی مفیدی به مدیریت جلسه ارائه می دهند. یکی از آنها **req.session.regenerate()** است که برای ایجاد یک شناسه جلسه جدید استفاده می شود و به شرح زیر استفاده می شود:

```
req.session.regenerate( function (err) {  
  // new session identifier has been created  
  // the req.session object has been re-instantiated  
  // with new values  
});
```

Reference for secure session configuration

```
var session = require( 'express-session' );  
  
app.use(session({  
  name : 'REPLACE_WITH_UNIQUE_NAME' ,  
  secret : 'REPLACE_WITH_UNIQUE_SECRET' ,  
  cookie : {  
    maxAge : null ,  
    httpOnly : true ,  
    secure : true  
  }  
}));
```

Summary

در این فصل، بهترین شیوه های ضروری مدیریت جلسه مانند موارد زیر را آموختیم:

- انتقال اطلاعات کوکی فقط از طریق اتصالات **HTTPS**
- جلوگیری از دسترسی به اطلاعات کوکی از زمان اجرای جاوا اسکریپت
- پنهان کردن نام کوکی برای پنهان کردن پشته برنامه وب شما

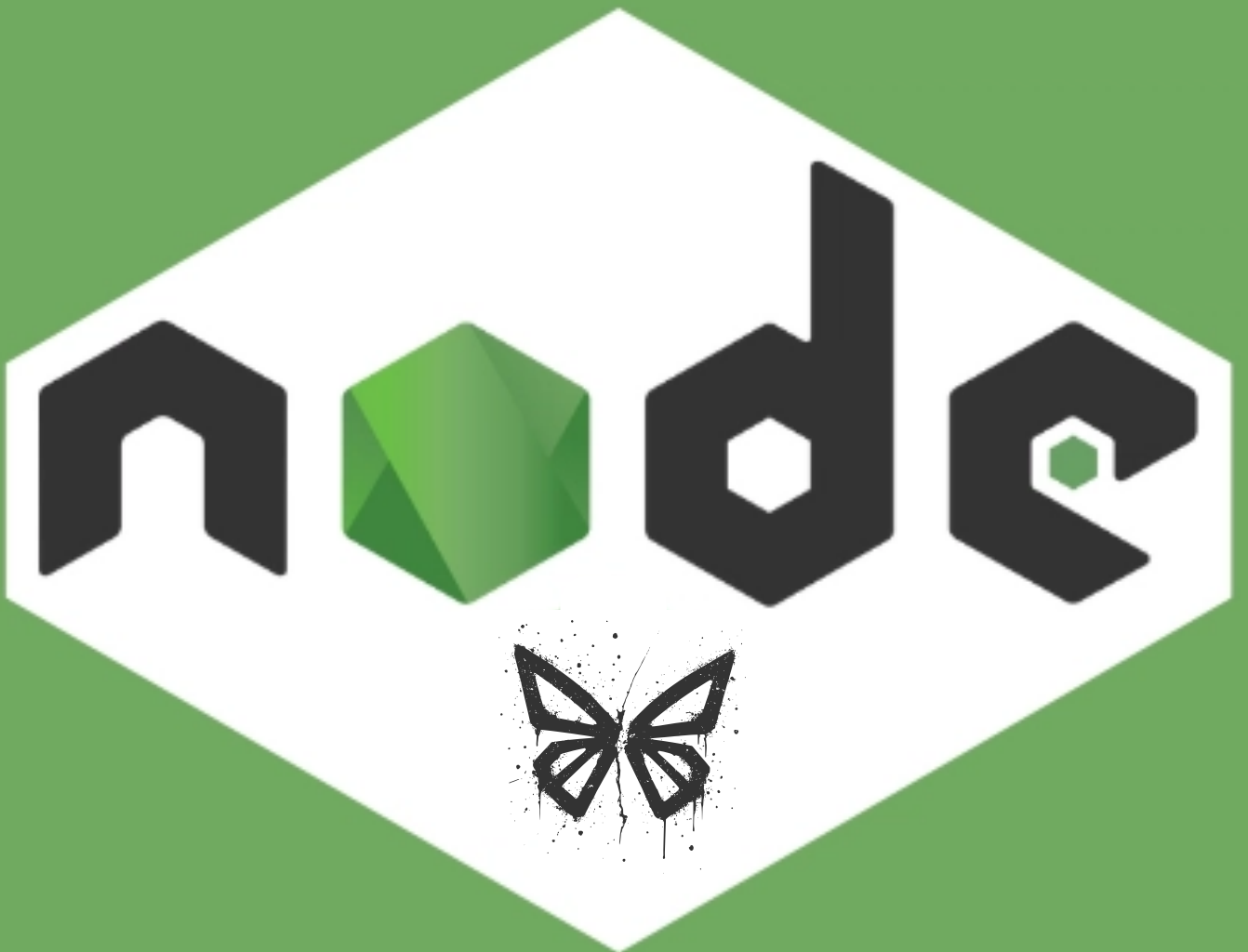




Console.log(`

Chapter three

`)





Chapter three

Hardening ExpressJS

اگرچه **ExpressJS** یک کتابخانه بسیار محبوب و کامل برای یک چارچوب برنامه وب است، اما هنوز هم می‌تواند فراتر از گزینه‌های پیش‌فرضی که استفاده می‌کند و مواردی که توسط افزونه‌های میان‌افزار مرتبط با آن استفاده می‌شوند، تنظیم شود. **ExpressJS** مورد اعتماد بسیاری از کاربران برای اجرا در سایت‌های عملیاتی است و توجه بیشتر به جزئیات و نحوه تقویت آن برای اطمینان از داشتن یک راه‌اندازی مقاوم در برابر امنیت بسیار مهم است.

Security Through Obscurity

ExpressJS از استانداردهای وب سرورهای **HTTP** پیروی می‌کند و به همین دلیل به طور پیش‌فرض هدر **X-Powered-By** را ارسال می‌کند که به هر درخواست وب گزارش می‌دهد کدام وب سرور در حال پردازش درخواست است. افشای چنین اطلاعاتی برای مهاجمان با آغوش باز مورد استقبال قرار می‌گیرد، زیرا آنها دانش خود را در مورد چارچوبی که شما استفاده می‌کنید به دست آورده‌اند و می‌توانند درخواست خود را بر حملات خاص **ExpressJS** متمرکز کنند. یکی از اولین، ساده‌ترین و آسان‌ترین اقدامات مقاوم‌سازی برنامه‌های وب، حذف این هدر است:

```
var express = require( 'express' );
var app = express();
app.disable( 'x-powered-by' );
```





Brute-Force Protection

حملات **Brute-force** ممکن است توسط یک مهاجم برای ارسال مجموعه‌ای از جفت‌های نام کاربری/رمز عبور به نقاط انتهایی **REST** شما از طریق **POST** یا **API RESTful** دیگری که برای پیاده‌سازی آنها باز کرده‌اید، استفاده شود. چنین حمله دیکشنری بسیار سراسر است و آسان برای اجرا است و می‌تواند در هر بخش دیگری از **API** یا مسیریابی صفحه شما، غیرمرتبط با ورود به سیستم، انجام شود. یک مورد استفاده رایج جایی است که ممکن است یک رابط مدیریتی در مسیر **admin** داشته باشید و یک مهاجم ممکن است سعی کند درخواست‌های خودکار را با توکن‌های مختلف، شناسه کوکی‌های مختلف و غیره برای ورود به آنجا ارسال کند. برای کمک به کاهش و محدود کردن درخواست‌های ارسالی به برنامه وب شما، می‌توانیم از کتابخانه‌ای به نام **express-limiter** استفاده کنیم که پیکربندی بسیار انعطاف‌پذیری را برای ادغام در یک برنامه **ExpressJS** ارائه می‌دهد.

پیش‌نیاز استفاده از **express-limiter** این است که

برای اتصال و مدیریت محدودیت‌هایی که به ازای هر درخواست اعمال می‌کند، به یک مخزن داده **Redis** نیاز دارد.

نصب **express-limiter** و به‌روزرسانی فایل **package.json** با ورودی آن:

```
npm install express-limiter --save
```





بیایید محدودیت زیر را ایجاد کنیم:

- محدود کردن انواع درخواستها (GET، POST، PUT و غیره) به مسیر /login
- محدود کردن درخواستها بر اساس آدرس IP ورودی
- مجاز کردن مجموعاً ۲۰ درخواست در ساعت

```
var express = require( 'express' );
var limiter = require( 'express-limiter' );
// create a redisClient object with default
//connection in formation
var redisClient = require( 'redis' ).createClient();
// instantiate an express's app variable
var app = express();
// bind the limiter object the express app object
//and pass the redisClient
// object so it knows how to persist the imposed limits
var limits = limiter(app, redisClient);
/**
 * Configuring the following limits:
 * - Limit all type of requests (GET, POST, PUT, etc)
to the /login path
 * - Limit the requests based on the incoming IP address
 * - Allow a total of 20 requests per hour
 */
limits({
  path : '/login' ,
  method : 'all' ,
  lookup : [ 'connection.remoteAddress' ],
  total : 20 ,
  expire : 1000*60*60
});
app.get( '/login' , function (req, res, next) {
  // we send a dummy object for every request being made
  // to /login path
  res.status( 200 ).send({ 'login' : 'ok' });
});
```





کلید جستجوی ارسالی به شیء **limits** بسیار انعطاف‌پذیر است و به طور پیش‌فرض با هر گزینه شیء/ویژگی در شیء **request** اکسپرس‌جی‌اس مطابقت دارد. فهرست کوتاهی از ویژگی‌های مفید و آموزنده شیء **request** عبارتند از:

headers.host •

headers.user-agent •

headers.accept-language •

headers.x-forwarded-for •

connection.remotePort و **connection.remoteAddress** •

url •

method •

path •

query •

protocol •

xhr •

ip •

معمولاً میان‌افزارهای دیگری مانند **passport** که قابلیت‌های احراز هویت و مجوزدهی را ارائه می‌دهند، شیء **req** را با داده‌های بیشتری مانند اطلاعات کاربر هنگام ورود به سیستم گسترش می‌دهند. بنابراین گزینه دیگری برای محدود کردن درخواست‌ها بر اساس شناسه کاربر: **user.id** یا بر اساس نام کاربری: **user.username** است.

اطلاعات بیشتر در مورد نحوه محدود کردن درخواست‌ها بر اساس روش‌های دیگر در صفحه گیت‌هاب **express-limiter** موجود است.





Advanced Functionality Limiting

می‌توانید تنظیمات بیشتری را در لایه وب سرور خود انجام دهید تا مسیرهای حمله احتمالی را بیشتر ببندید. کتابخانه‌ای از یاهو! امکان هشدار دادن به این تنظیمات را فراهم می‌کند و **limit*** یا **node-limits** نامیده می‌شود، اگر می‌خواهید در صفحه **GitHub** آنها بیشتر تحقیق کنید.

به عنوان مثال، اگر برنامه وب شما هیچ درکی از آپلود فایل ندارد، می‌توانید این نوع ارسال فرم را به طور کامل غیرفعال کنید.

اگر اجازه آپلود فایل را می‌دهید، می‌توانید حداکثر اندازه درخواستی را که برای پردازش به شما ارسال می‌شود، محدود کنید. این موارد می‌تواند به کاهش هرگونه تلاش برای کند کردن پهنای باند و خط لوله شبکه و کاهش فضای دیسک موقت یا هرگونه عملیاتی که توسط برنامه وب شما برای پردازش آپلود فایل انجام می‌شود، کمک کند.

از دیگر تنظیماتی که این کتابخانه ارائه می‌دهد، پیکربندی آستانه‌های زمان انتظار است.

به عنوان مثال، ممکن است بخواهید یک زمان انتظار جهانی برای اتصالات ورودی تنظیم کنید تا مطمئن شوید که یک مهاجم سعی نمی‌کند سوکت‌های زیادی را روی سیستم عامل وب سرور شما باز نگه دارد.

نصب کتابخانه **limits** و به‌روزرسانی **package.json**:

`npm install limits --save`





ایجاد پیکربندی محدودیت زیر:

- غیرفعال کردن آپلود فایل
- محدود کردن همه درخواست‌ها به مجموع ۲ مگابایت
- تنظیم زمان انقضای سراسری برای اتصالات ورودی به ۱ دقیقه

```
var nodeLimits = require( 'limits' );
app.use(nodeLimits({
  file_uploads : false ,
  post_max_size : 2000000 ,
  inc_req_timeout : 1000*60*60
}));
```

body-parser middleware

میان‌افزار **body-parser** چارچوب وب **ExpressJS** را با پشتیبانی از درخواست‌های ارسالی و تجزیه داده‌های بدنه **HTTP** برای انواع داده‌های رایج مانند **JSON** تقویت می‌کند. **body-parser** بسیار محبوب است و گزارش شده است که ماهانه بیش از پنج میلیون دانلود دارد. همانطور که در کتابخانه **limits** قبلی مشاهده شد، می‌توان اندازه درخواست ورودی را محدود کرد تا باعث فشار بر **CPU** سرور برای تجزیه شیء **body** نشود. در حالی که کتابخانه **limits** ممکن است برای برخی از برنامه‌های وب بیش از حد باشد، **body-parser** بسیار رایج است و می‌توان آن را طوری تنظیم کرد که درخواست‌های خاصی را که مدیریت می‌کند محدود کند تا امنیت را فراهم کند.

برای نصب و به‌روزرسانی فایل **package.json** با کتابخانه **body-parser**:

npm install body-parser --save





محدود کردن اندازه درخواست‌ها به شرح زیر عمل می‌کند:

```
var bodyParser = require( 'body-parser' );  
// instruct bodyParser to parse JSON data and populate  
// the body data payload  
// in the request object *req* with an actual JSON object  
app.use(bodyParser.json({  
  limit : '1mb'  
}));
```

پس از تعیین محدودیت، وقتی درخواست ورودی بزرگتر از حد مجاز باشد، **ExpressJS** خطای موجودیت درخواست بیش از حد بزرگ (**error request entity too large**) را که یک پاسخ استاندارد **HTTP** است، منتشر می‌کند و با کد **HTTP 413** پاسخ می‌دهد. اعمال محدودیت‌ها به یاد داشته باشید که **body-parser** فقط ارسال فرم‌های غیر چند قسمتی را مدیریت می‌کند، بنابراین تنظیم این محدودیت تاثیری بر آپلود فایل‌های ارسالی به برنامه وب شما نخواهد داشت.

Summary

در این فصل، برخی از ابزارها و پیکربندی‌هایی را که به مقاومت‌سازی یک برنامه وب **ExpressJS** کمک می‌کنند، بررسی کردیم، مانند پیاده‌سازی محدود کردن نرخ برای کاهش سیل درخواست‌ها، پیاده‌سازی سهمیه برای آپلود فایل و به‌طور کلی پنهان کردن جزئیات **ExpressJS** به‌عنوان چارچوب برنامه.

مستندات **ExpressJS** همچنین جزئیات و بینش‌های بیشتری در مورد بهترین شیوه‌های امنیتی که به مقاومت‌سازی و ایمن‌سازی برنامه‌های **ExpressJS** کمک می‌کنند، ارائه می‌دهد. **Product Best Practices** برخی از این اطلاعات را بررسی می‌کند و احتمالاً در آینده به‌روزرسانی‌هایی دریافت خواهد کرد، بنابراین این همیشه منبع خوبی برای پیگیری است.

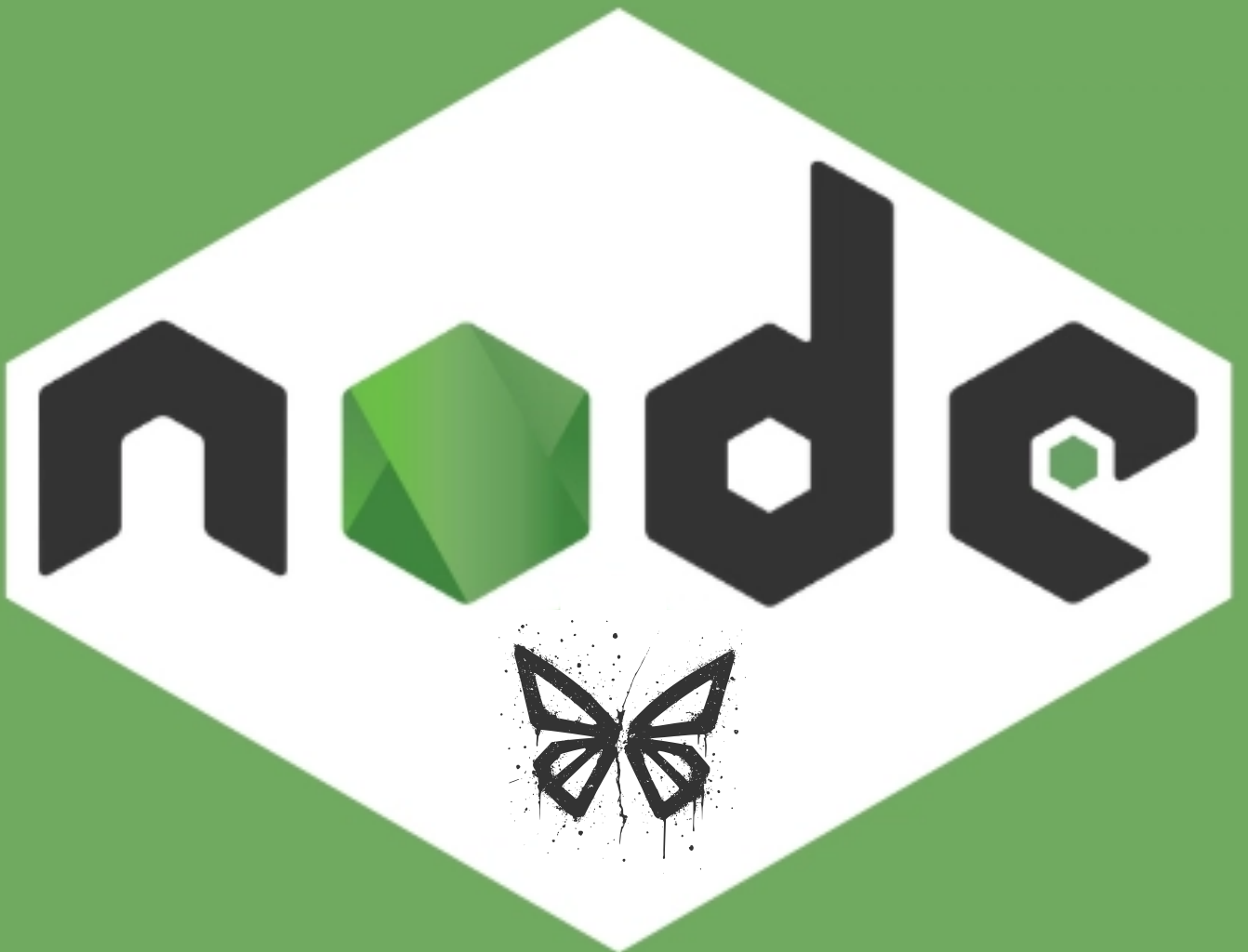




Console.log(`

Chapter four

`)





Chapter four

Cross-Site Request Forgery (CSRF)

CSRF که نام خود را از حمله‌ای که انجام می‌دهد گرفته است، قربانی را فریب می‌دهد تا ناخواسته درخواست‌هایی را به سیستمی ارسال کند که کاربر به آن دسترسی دارد و احتمالاً از قبل در آن وارد سیستم شده است. معمولاً این حملات ذاتاً هدفمند هستند زیرا مهاجم باید یک **CSRF** بسازد و کاربر را فریب دهد تا عملی را روی سیستم دیگری غیر از سیستم مهاجم انجام دهد. بنابراین، مهاجم احتمالاً از سیستم هدفی که **CSRF** برای آن ساخته شده است، آگاهی قبلی دارد.

نکته: **CSRF** که با نام‌های دیگری نیز شناخته می‌شود، نام‌های دیگری نیز دارد که سایر فروشندگان و جوامع از آنها استفاده می‌کنند، از جمله: **One-Click attack** توسط میکروسافت، **Session Riding**، و اغلب حتی به صورت اختصاری **CSRF** نیز نامیده می‌شود.

The Risk

یک مورد استفاده مثالی، جایی است که یک برنامه وب آسیب‌پذیر ممکن است فرمی داشته باشد که از روش **GET** برای ارسال استفاده می‌کند و بنابراین داده‌های فیلد ورودی خود را از پارامترهای پرس و جو دریافت می‌کند. در این حالت، می‌توان به راحتی کاربر را از چندین طریق فریب داد تا آن روش **GET** را در فرم **HTML** ارسال کند.





یک ایمیل یا وبسایت جعلی که کاربر را به کلیک روی یک لینک ترغیب می‌کند یا حتی سعی می‌کند تصویری را ارائه دهد که منجر به ارسال این فرم شود. برای مثال:

```
<img src= "http://target-web-application.com/updateEmailAddress.php?email=attacker@domain.com" />
```

وقتی سرویس ایمیل کاربر سعی می‌کند این قطعه **HTML** را تفسیر کرده و تگ تصویر را رندر کند، در واقع باعث می‌شود مرورگر آن درخواست را از طرف کاربر انجام دهد. اگر کاربر وارد سیستم شده باشد، این مثال از متد **GET** با نام **FORM** ارسال می‌شود و در نتیجه آدرس ایمیل کاربر تغییر می‌کند.

یک لینک ساده همچنین می‌تواند کاربر را به کلیک روی آن سوق دهد - بدون اینکه بداند این لینک در واقع چه چیزی را به او نشان می‌دهد:

```
<a href= "http://target-web-application.com/updateEmailAddress.php?email=attacker@domain.com" > Read More </a>
```

به‌روزرسانی پیاده‌سازی فرم برای استفاده از درخواست‌های **POST** یا **PUT** هیچ سطح امنیتی بالاتری را برای پیاده‌سازی فرم‌های امن فراهم نمی‌کند. برخی از نمونه‌های حمله به این فرم‌ها عبارتند از:

مهاجم وبسایتی را کنترل می‌کند که می‌تواند حاوی یک فرم ارسالی ساده با مسیر اقدام تنظیم‌شده برای برنامه وب آسیب‌پذیر هدف باشد. به عنوان مثال:

```
<form action= "http://target-web-application.com/updateEmailAddress.php"
method="post" >
<input type= "hidden" name= "email"
value= "attacker@domain.com" />
<input type= "submit" value= "Continue" />
</ form>
```





وقتی کلیک شود، مرورگر درخواستی را به مسیر مشخص شده در ویژگی **action** ارسال می کند که شامل یک جفت کلید/مقدار مخفی برای به روزرسانی آدرس ایمیل کاربر است.

یک مهاجم همچنین می تواند از کد جاوا اسکریپت برای ارسال مثال **FORM** با متد **POST** قبلی به همراه رویداد بارگذاری صفحه استفاده کند، به طوری که کاربر هیچ چیزی در مورد آن نگوید (مگر اینکه جاوا اسکریپت کاربر غیرفعال باشد):

`<body onLoad= "document.forms[o].submit()" >`

مورد دیگری که می تواند توسط کاربر مورد سوءاستفاده قرار گیرد، ایجاد یک درخواست **AJAX** است که مرورگر را برای پردازش و ارسال این درخواست تحریک می کند:

```
<script type= "text/javascript" >
var xhr = new XMLHttpRequest();
xhr.open( "POST" , "http://target-web-application.com/updat\
eEmailAddress.php" );
xhr.setRequestHeader( "Content-Type" , "application/x-www-f\
orm-urlencoded" );
xhr.send( "email=attacker@domain.com" );
< /script>
```

این درخواست **AJAX** همچنین می تواند به طور خودکار هنگام بارگذاری صفحه وب مهاجم توسط کاربر و با اتصال ساده آن به رویداد **onLoad** اجرا شود.

خطر بزرگتر جایی است که برنامه وب هدف در واقع میزبان کد **CSRF** است. به عنوان مثال، اگر برنامه وب امکان ارسال نظرات متنی غنی، انجمن ها یا هرگونه ورودی غنی دیگر از کاربر را فراهم کند، آنگاه این برنامه هدف بسیار جذابی برای مهاجمان خواهد بود تا حمله **CSRF** را در آنجا تزریق کنند و این آسیب پذیری را به میزان قابل توجهی افزایش دهند.





The Solution

همانطور که هنگام بررسی خطرات احتمالی **CSRF** توضیح داده شد، هرگونه تلاش برای مقاومسازی یک موجودیت **HTMLFORM** بیپهوده است و صرفاً برای مهاجم آزاردهنده است تا بتواند راهحلی پیدا کند، اما چنین تلاش‌هایی قطعاً راهحل نیستند.

برای روشن شدن موضوع، بیایید بررسی کنیم که چه روش‌های اشتباهی برای رسیدن به راهحل **CSRF** وجود دارد:

۱. تغییر ویژگی متد **FORM** از **GET** به **POST**

۲. تغییر مقدار ویژگی اکشن **FORM** از **HTTP** به **HTTPS** یا به‌روزرسانی **URI** به یک **URL** کامل.

۳. منسوخ کردن تمام عناصر **FORM** و تبدیل آنها به **API**

۴. افزودن اقدامات بیشتر برای تأیید ارسال **FORM** مانند پنجره‌های بازشو یا فرم‌های ثانویه

۵. ذخیره هرگونه اطلاعات پنهان در داخل یک کوکی برای تأیید درخواست‌ها

همه موارد فوق نمونه‌هایی از کارهایی هستند که نباید برای محافظت در برابر آسیب‌پذیری‌های **CSRF** انجام داد، زیرا برخی از آنها یا اشتباه هستند، توهمی از یک راهحل ارائه می‌دهند، یا به سادگی مشکل را به طور کامل حل نمی‌کنند.





CSRF Tokens

روش ترجیحی برای محافظت در برابر حملات **CSRF**، تولید یک توکن است که در اصل یک رشته تصادفی و غیرقابل حدس است و برای هر عملی که توسط کاربر انجام می‌شود، تولید می‌شود. با هر عمل کاربر، این توکن بین آنچه کاربر ارسال کرده و آنچه سرور انتظار دارد (توکن قبلی که تولید شده است) مقایسه می‌شود. در مواردی که مقایسه با شکست مواجه می‌شود، توکن‌های **CSRF** با هم مطابقت ندارند و عمل نیز به دلیل یک حمله احتمالی رد می‌شود. توکن‌های **CSRF** را می‌توان با عدم استفاده از یک توکن واحد برای کل جلسه کاربر، که ممکن است در معماری برنامه تک صفحه‌ای (**SPA**) رایج باشد، ایمن‌تر کرد، بلکه می‌توان توکن‌های جدیدی تولید کرد و برای هر عمل فرم ارسالی یا عمل مشابه کاربر که انجام می‌شود، با آنها مقایسه کرد.

کاراکترهای ناخوانا که ما آنها را **CAPTCHA** می‌نامیم

مفهوم **CAPTCHA** در ابتدا برای کاهش هزینه‌های کاربران، ربات‌ها و خزش خودکار وب معرفی شد. این یک راه حل ممکن برای افزایش امنیت فرم‌ها و اقدامات است، اما کاربرپسند نیست. ضمناً، آیا می‌دانستید که معنی **CAPTCHA** عبارت است از: **Completely Automated Public Turing Test To Tell Computers and Humans Apart** خوشبختانه ما یک مخفف برای آن داریم.

CSRF Tokens Implementation

روش‌های مختلفی برای پیاده‌سازی توکن **CSRF** وجود دارد و این روش‌ها متفاوت هستند و به معماری یک برنامه وب بستگی دارند. اصول اولیه مقایسه یک توکن تولید شده با توکن دریافتی در ورودی، یکسان باقی می‌ماند، فقط نحوه تحویل و تبادل توکن بین سرور و کلاینت تغییر می‌کند.

Request Body

با پیاده‌سازی بدنه درخواست، سرور توکن **CSRF** را تولید می‌کند که سپس در لایه نمایش برای قرار دادن روی فرم‌ها به عنوان یک عنصر ورودی پنهان استفاده می‌شود. هنگامی که کاربر فرم را ارسال می‌کند، عنصر ورودی پنهان با مقدار **CSRF** نیز به عنوان بخشی از بدنه درخواست ارسال می‌شود که سپس توسط سرور دریافت می‌شود و سرور می‌تواند توکن **CSRF** را از ورودی با توکنی که با نمای صفحه تولید شده





است مقایسه کند. این رویکرد نیاز به مدیریت توکن **CSRF** در هر صفحه دارد، بنابراین باید هنگام ایجاد صفحه وب از قبل برنامه‌ریزی و طراحی شود، از این رو از دیدگاه معماری برنامه، آن را دست و پا گیر می‌کند. علاوه بر این، در برنامه‌های تک صفحه‌ای (**SPA**) معماری، بارگذاری یک صفحه را دیکته می‌کند، بنابراین سرور در واقع فقط یک بار توکن **CSRF** را تولید می‌کند و آن توکن باید با هر درخواست بعدی که توسط برنامه وب ارسال می‌شود مقایسه شود.

CSRF Cookie

اکثر برنامه‌های وب برای ذخیره‌سازی سمت کلاینت و حفظ یک جلسه در یک پروتکل **HTTP** بدون وضعیت، نیاز به استفاده از کوکی‌ها دارند. با استفاده از مخزن کوکی، برنامه وب می‌تواند یک کوکی **CSRF** با مقدار توکن تنظیم کند. در این مرحله، تمام کوکی‌های برنامه وب با هر درخواستی که کاربر به سرور ارسال می‌کند، از جمله کوکی توکن **CSRF** که قبلاً تنظیم شده است، ارسال می‌شوند. این به خودی خود هیچ محافظتی ایجاد نمی‌کند، اما پیاده‌سازی چیزی که به عنوان مکانیزم «ارسال دوگانه کوکی‌ها» شناخته می‌شود، این کار را انجام می‌دهد. کوکی‌های ارسال دوگانه با ارسال مقادیر تصادفی هم در خود درخواست و هم در یک مقدار کوکی کار می‌کنند، که سپس سرور آن را مقایسه و تأیید می‌کند.

CSRF Token Header

تکیه بر یک هدر **HTTP** سفارشی برای تبادل اطلاعات در مورد توکن **CSRF**، سطح بالایی از امنیت محسوب می‌شود، زیرا مهاجم را ملزم می‌کند که واقعاً بتواند ترافیک شبکه را "استخراج" کند یا یک حمله مرد میانی (**MITM**) انجام دهد. در مواردی که هم سرور و هم کلاینت از هدر توکن **CSRF** خاص استفاده می‌کنند، وقتی سرور توکن را تولید می‌کند، با این توکن ویژه پاسخ می‌دهد و وقتی کلاینت پاسخ را دریافت می‌کند، می‌تواند هدر **HTTP** را برای مقدار توکن تجزیه کند و سپس آن را به عنوان همان هدر **HTTP** که سرور انتظار دارد، به سرور ارسال کند.





CSRF Libraries – under the hood

بررسی کد منبع کتابخانه **csrf**، بینش بیشتری در مورد نحوه مدیریت توکن توسط یک کتابخانه **CSRF** و محل انتظار آن، به ترتیب اولویت، ارائه می‌دهد:

```
/**
 * Default value function, checking the `req.body`
 * and `req.query` for the CSRF token.
 *
 * @param {IncomingMessage} req
 * @return {String}
 * @api private
 */
function defaultValue(req) {
  return (req.body && req.body._csrf)
    || (req.query && req.query._csrf)
    || (req.headers[ 'csrf-token' ])
    || (req.headers[ 'xcsrf-token' ])
    || (req.headers[ 'x-csrf-token' ])
    || (req.headers[ 'x-xcsrf-token' ]);
}
```

اگر مقایسه توکن با شکست مواجه شود، کتابخانه **csrf** خطایی را نمایش می‌دهد:

```
/**
 * Verify the token.
 *
 * @param {IncomingMessage} req
 * @param {Object} tokens
 * @param {string} secret
 * @param {string} val
 * @api private
 */
function verifytoken(req, tokens, secret, val) {
  // valid token
  if ( ! tokens.verify(secret, val)) {
    throw createError( 403 , 'invalid csrf token' , {
      code : 'EBADCSRFTOKEN'
    });
  }
}
```





به طور خلاصه، سروری که کتابخانه **csrf** را پیاده‌سازی می‌کند، همیشه به دنبال توکن **CSRF** در پارامترهای درخواست، بدنه داده یا در یک هدر **HTTP** خاص می‌گردد. این مقدار را دریافت کرده و آن را با مقداری که در ابتدا ذخیره کرده است مقایسه می‌کند تا تطابق را بررسی کند.

ExpressJS csrf Library

csrf یکی دیگر از میان‌افزارهای خانواده **ExpressJS** است که مکانیسمی برای مدیریت توکن‌های **CSRF** فراهم می‌کند. کتابخانه **csrf** از حافظه نشست سرور یا حافظه کوکی کلاینت برای ذخیره و مقایسه توکن **CSRF** استفاده می‌کند، بنابراین باید همراه با یکی از آنها استفاده شود. این فصل استفاده از هر دوی این گزینه‌ها را پوشش خواهد داد.

نصب **csrf** برای استفاده در یک پروژه **expressjs**:

```
npm install csrf--save
```

csrf With Cookies

حداقل نیاز برای **csrf**، کتابخانه **body-parser** برای دسترسی به داده‌ها از شیء **req** و سپس کتابخانه **session (express-session)** یا کتابخانه **cookie-parser** برای حفظ مقدار توکن **CSRF** است. برای شروع، تمام کتابخانه‌ها در کد مورد نیاز هستند:

```
var bodyParser = require('body-parser');  
var cookieParser = require('cookie-parser');  
var csrf = require('csrf');
```

در مرحله بعد، کتابخانه‌ها مقداردهی اولیه و پیکربندی می‌شوند:

```
// initialize the body-parser library  
app.use(bodyParser.urlencoded());  
  
// setup cookie information  
app.use(cookieParser('secretKey'));
```





فقط در این صورت، میان افزار **csrf** می تواند مقداردهی اولیه شود (این ترتیب میان افزار مهم است). **csrf** طوری پیکربندی شده است که از فضای ذخیره سازی کوکی ها استفاده کند و به عنوان بخشی از میان افزار برای یک شیء برنامه **ExpressJS** اضافه می شود:

```
var csrfToken = csrf({cookie : true});  
app.use(csrfToken);
```

از این مرحله به بعد، تنها کاری که باقی می ماند پیکربندی مسیر برای دسترسی به توکن **CSRF** است. با استفاده از یک مثال ساده که در آن یک **view** یک عنصر **HTML FORM** را رندر می کند، مسیر، توکن **CSRF** را به فیلد ورودی توکن **CSRF** مخفی ارائه می دهد.

```
app.get( '/login' , function (req, res, next) {  
  res.render( 'login' , {  
    csrfToken : req.csrfToken()  
  })  
});
```

نمای مربوط به مسیر **/login** یک فرم ساده است که از متغیر **csrfToken** در قالب به عنوان بخشی از یک فیلد ورودی پنهان استفاده می کند:

```
<form action= "/login" method= "post" >  
<input type= "hidden" name= "_csrf" value= "<%= csrfToken %>" >  
<button type= "submit" >Submit</ button>  
</ form>
```

این ویو هنگام ارسال، به مسیر **/login** ارجاع می دهد و آن را به عنوان یک درخواست **HTTP** از نوع **POST** ارسال می کند. مسیر نمونه ای که در زیر برای مدیریت ارسال فرم اضافه شده است، تنها در صورتی که توکن **CSRF** اعتبارسنجی را با موفقیت پشت سر بگذارد، پاسخ **HTTP 200** را برمی گرداند:

```
app.post( '/login' , function (req, res, next) {  
  res.status( 200 ).send({  
    'csrf' : 'ok'  
  });  
});
```





csrf With Session

کتابخانه‌های زیر ابتدا در کد مورد نیاز هستند:

```
var bodyParser = require( 'body-parser' );
var session = require( 'express-session' );
var csrf = require( 'csrf' );
```

از آنجا که ما از **session**ها برای ماندگاری استفاده می‌کنیم، در حالت ایده‌آل **session** باید پیکربندی شود و همچنین کتابخانه **body-parser** را مقداردهی اولیه می‌کنیم تا میان‌افزار **csrf** بتواند توکن مورد نیاز را در شیء **req** پیدا کند:

```
// configure session and cookie details
app.use(session({
  name : 'csrfExampleSession' ,
  secret : 'csrfSecretExampleKey' ,
  resave : true ,
  saveUninitialized : true ,
  cookie : {
    maxAge : null ,
    httpOnly : true ,
    secure : false
  }
}));

// initialize the body-parser library
app.use(bodyParser.urlencoded( ));
```

از این مرحله به بعد، جریان کار مشابه نحوه عملکرد پیکربندی **csrf** با کوکی‌ها است. تنها کاری که باقی می‌ماند پیکربندی مسیر برای دسترسی به توکن **CSRF** است. با استفاده از یک مثال ساده که در آن یک **view** یک عنصر **HTML FORM** را رندر می‌کند، مسیر، توکن **CSRF** را به فیلد ورودی توکن **CSRF** مخفی ارائه می‌دهد.





```
app.get( '/login' , function (req, res, next) {  
  res.render( 'login' , {  
    csrfToken : req.csrfToken()  
  })  
});
```

نمای مسیر **/login** یک فرم ساده است که از متغیر **csrfToken** در قالب به عنوان بخشی از یک فیلد ورودی پنهان استفاده می‌کند:

```
<form action= "/login" method= "post" >  
<input type= "hidden" name= "_csrf" value= "<%= csrfToken %>" >  
<button type= "submit" >Submit</ button>  
</ form>
```

این نما هنگام ارسال، به مسیر **/login** ارجاع می‌دهد و آن را به عنوان یک درخواست **HTTP POST** ارسال می‌کند. مسیر نمونه‌ای که در زیر برای مدیریت ارسال فرم اضافه شده است، تنها در صورتی که توکن **CSRF** اعتبارسنجی را با موفقیت پشت سر گذاشته باشد، پاسخ **HTTP 200** را دریافت کرده و برمی‌گرداند:

```
app.post( '/login' , function (req, res, next) {  
  res.status( 200 ).send({  
    'csrf' : 'ok'  
  });  
});
```

ExpressJS With lusca Library

lusca یک میان‌افزار امنیتی برنامه‌های وب است که در کنار بسیاری از ویژگی‌های دیگر که در فصل‌های قبلی به آنها پرداخته شد، امنیت توکن **CSRF** را نیز فراهم می‌کند و با چارچوب‌های برنامه‌های وب مانند **ExpressJS** ادغام می‌شود. مشابه **csrf**، **lusca** نیز برای ذخیره‌سازی و ماندگاری به یک میان‌افزار **session** یا **cookie** و همچنین میان‌افزار **body-parser** نیاز دارد. نصب **lusca** اگر این کار در فصل‌های قبلی انجام نشده باشد:

npm install lusca --save





lusca With Session

تنظیمات برنامه بسیار شبیه به **csrf** است، بنابراین در اینجا به عنوان یک نمونه کد منبع کامل ارائه شده است و برای ارجاعات خاص هر بخش از کد، می‌توان به بخش‌های **csrf** مراجعه کرد.

```
var express = require( 'express' );
var session = require( 'express-session' );
var bodyParser = require( 'body-parser' );
var lusca = require( 'lusca' );

// configure session and cookie details
app.use(session({
  name : 'luscaCsrfExampleSession' ,
  secret : 'luscaCsrfSecretExampleKey' ,
  resave : true ,
  saveUninitialized : true ,
  cookie : {
    maxAge : null ,
    httpOnly : true ,
    secure : false
  }
}));

app.use(bodyParser.urlencoded());
var luscaCsrf = lusca({csrf : true});
app.use(luscaCsrf);

app.get( '/login' , function (req, res, next) {
  res.render( 'login' , {
    csrfToken : req.csrfToken()
  })
});

app.post( '/login' , function (req, res, next) {
  res.status( 200 ).send({
    'csrf' : 'ok'
  });
});
```

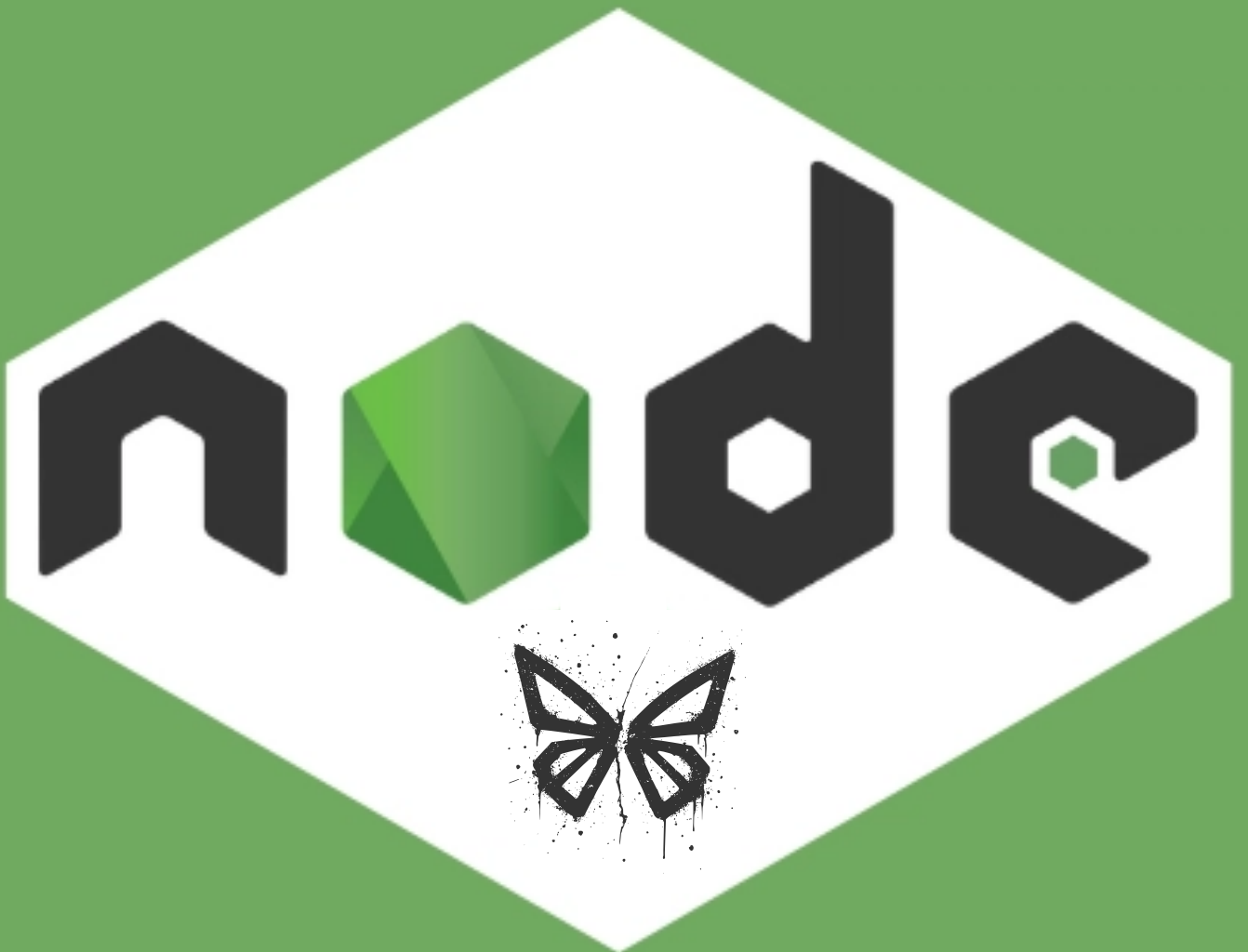
Summary

پیاده‌سازی امنیت توکن **CSRF** برای کاهش حملات **CSRF** یک لایه حیاتی و اساسی برای ایمن‌سازی یک برنامه وب سمت کلاینت است. در حالی که ما **csrf** و **lusca** را به عنوان کتابخانه‌های مناسب برای ادغام در یک برنامه **Node.js** بررسی کردیم، پیاده‌سازی‌های دیگری هم در سمت کلاینت و هم در سمت سرور برای کمک به محافظت از کاربران نهایی وجود دارد. یک مثال، پشتیبانی داخلی **AngularJS** از مکانیسم توکن **CSRF** است که می‌توان در مستندات سرویس **https** آنها بیشتر به آن پرداخت.





Console.log(` Chapter five `)





Cross-Site Scripting (XSS)

حمله اسکریپت نویسی بین سایتی (XSS) با توانایی مهاجم در تزریق هر نوع اسکریپتی مانند **flash**، **HTML** یا جاوا اسکریپت به یک برنامه وب مشخص می‌شود که برای اجرا و رندر شدن در برنامه‌ای که صفحه را ارائه می‌دهد، در نظر گرفته شده‌اند. برنامه وب ناخواسته کد اسکریپتی را که توسط مرورگر اجرا می‌شود، ارائه می‌دهد و از این رو کاربر را در برابر سرقت داده‌ها و هر سطح دسترسی که اسکریپت مجاز است، آسیب‌پذیر می‌کند. منبع آسیب‌پذیری **XSS** در یک برنامه وب نهفته است که اجازه می‌دهد کد مخرب به عنوان بخشی از صفحه وب که به کاربر ارائه می‌شود، تزریق و ارزیابی شود و سپس همان کد مخرب به دلیل عدم توانایی برنامه وب در فیلتر کردن و پاکسازی خروجی توسط مرورگر اجرا می‌شود.

برای بررسی یک مثال از یک مورد استفاده ساده از یک صفحه وب که نام کاربر را از پارامتر پرس و جو می‌خواند `http://example.com/profile?name=John`، و سپس از این پارامتر برای نمایش نام کاربر در صفحه پروفایل استفاده می‌کند:

```
<div>
<h2>{{name}}</h2>
</div>
```

چه اتفاقی می‌افتد اگر کسی رشته **John** را با کد جاوا اسکریپت جایگزین کند؟

`http://example.com/profile?name=<script>alert( "xss" ) </script>`

اگر هیچ عملیات **escape** رشته‌ای روی پارامتر **name** در سطح خروجی انجام نشود، یا حداقل مقداری پاکسازی روی داده‌هایی که انتظار می‌رود پارامتر **name** معتبر باشد، انجام شود، در صفحه رندر شده، کد جاوا اسکریپت برای کاربر چاپ می‌شود و مرورگری که این اسکریپت را رندر می‌کند، یک کادر محاوره‌ای هشدار نمایش می‌دهد:

```
<div>
<h2><script>alert( "xss" )</ script></ h2>
</ div>
```





The Risk

آسیب‌پذیری‌های **XSS** را می‌توان در یکی از دسته‌های زیر طبقه‌بندی کرد:

- **Stored/Persistent XSS** – همانطور که از نامش پیداست، حمله **XSS** ذخیره‌شده زمانی است که کد **XSS** مخرب به برنامه وب تزریق می‌شود و در حافظه پایداري که برنامه پیاده‌سازی می‌کند، ذخیره می‌شود. به عنوان مثال، اگر برنامه وب اجازه ارسال نظر را بدهد و ورودی نظرات اعتبارسنجی یا پاکسازی نشود، مهاجم می‌تواند کد **XSS** مخرب را به عنوان بخشی از نظر تزریق کند. به دلیل ذخیره شدن نظر در برنامه وب، هنگامی که صفحه، نمای نظر را برای هر کاربری نمایش می‌دهد، کاربر را نیز در معرض این حمله قرار می‌دهد.
- **Reflected XSS** – این نوع حمله **XSS** نتیجه مشابهی برای کاربر نهایی دارد، اما از **XSS** ذخیره شده شدت کمتری دارد زیرا برنامه وب همه کاربران را به طور یکسان در معرض کد مخرب قرار نمی‌دهد، بلکه در عوض، مهاجم می‌تواند یک لینک مخرب ایجاد کند که وقتی کاربر فریب می‌خورد تا آن را مشاهده کند، درخواست، کد مخرب را به برنامه وب تزریق می‌کند و سپس در مرورگر کاربر نمایش داده می‌شود.

برای مثال، تصور کنید یک عبارت جستجو به صورت زیر انجام می‌شود:

`http://example.com/search?movieName=Inception` که در آن یک برنامه وب رایج از پارامتر **movieName** برای اطلاع‌رسانی به کاربر در مورد آنچه جستجو کرده است، استفاده می‌کند. اگر برنامه وب ناامن باشد، حمله **XSS** انعکاسی می‌تواند رخ دهد و مهاجم می‌تواند مقدار پارامتر **movieName** را با یک کد اجرایی جاوا اسکریپت مخرب جایگزین کند.





• **DOM-based XSS** - ماهیت این حمله **XSS** در کد برنامه وب نهفته است که از روش‌های **DOM** استفاده می‌کند که به ورودی ناامن کاربر متکی هستند. مرورگرها رایج‌ترین شیء سند را ارائه می‌دهند که امکان تعامل با ساختار صفحه وب و درخواست وب فعلی ایجاد شده را فراهم می‌کند. ویژگی‌های آسیب‌پذیر شیء سند عبارتند از **document.location** یا **document.documentURI** که چند نمونه از آنها هستند. اگر یک برنامه وب از یکی از این ویژگی‌ها به صورت ناامن برای تجزیه داده‌های درخواست ایجاد شده و سپس استفاده از آن در صفحه وب استفاده کند، مهاجم می‌تواند درخواست را درست مانند حمله **XSS** منعکس شده تغییر دهد و بنابراین بر ساختار **DOM** تأثیر بگذارد و مرورگر کاربر را در معرض اجرای کد مخرب قرار دهد.

مرور مختصری بر منابع داده‌های ورودی غیرقابل اعتماد که ممکن است در برابر حملات **XSS** آسیب‌پذیر باشند:

- رشته پرس‌وجو و پارامترها (**Query string and parameters**) - اینها رایج‌ترین تزریق‌های ورودی هستند و شامل هر پارامتر **GET**، خود **URL** یا بخش‌هایی از آن، و داده‌های عمومی **POST** یا سایر روش‌های **HTML** می‌شوند.
- کوکی‌ها (**Cookies**) - حتی کوکی‌ها نیز ممکن است حاوی داده‌هایی باشند که یک مهاجم توانسته به نحوی کد مخرب را به آنها تزریق کند و باید با احتیاط با آنها رفتار شود.





Variations of XSS syntax

جاوا اسکریپت که پرکاربردترین زبان برای کدهای مخرب است، می‌تواند به روش‌هایی غیر از تگ‌های رایج «**script**» نمایش داده و منتقل شود. برای مثال، **XSS** با استفاده از ویژگی‌های رویداد یا **event** در **HTML** می‌تواند **HTML** را که از رویدادهای **DOM** پشتیبانی می‌کند تا به عنوان یک ویژگی به موجودیت‌های **HTML** اختصاص داده شوند. وقتی این رویدادها اختصاص داده می‌شوند، امکان اجرای کد جاوا اسکریپت را فراهم می‌کنند که نیازی به قرار گرفتن در تگ‌های `<script> ... </script>` ندارد:

```
<button onClick= "alert('xss')" >Submit</ button>
```

خطری که این حمله ایجاد می‌کند این است که برنامه‌های وب که سعی در استفاده از **denylist** یا فیلتر کردن تگ‌های **HTML** به اصطلاح خطرناک مانند تگ‌های اسکریپت دارند، در این حالت شکست می‌خورند، زیرا مهاجم می‌تواند با قرار دادن کد جاوا اسکریپت به عنوان بخشی از رویدادهای مجاز **DOM**، آن را به صفحه تزریق کند.

منابع بیشتر برای آشنایی با تزریق مرتبط با **XSS**:

XSS syntax variations - ویکی **OWASP** شامل یک برگه تقلب جامع و بسیار دقیق برای دور زدن فیلتر **XSS** است که انواع مختلف تزریق‌های ممکن را که می‌تواند توسط یک مهاجم برای دور زدن کنترل‌های حفاظتی شما و موفقیت در یک حمله **XSS** به کار گرفته شود، نشان می‌دهد.

HTML5 Security - با توجه به مشخصات جدید **HTML5**، مرورگرها در حال پذیرش دستورالعمل‌ها، ویژگی‌ها و عناصر جدید هستند و این امر مسیرهای حمله جدیدی را معرفی می‌کند که برخی از آنها مربوط به **XSS** هستند. **†html5sec** یک وبسایت مرجع خوب برای به‌روز ماندن در مورد چنین آسیب‌پذیری‌هایی مربوط به پذیرش ناامن ویژگی‌های **HTML5** است.





The Solution

آسیب‌پذیری‌های **XSS** با سوءاستفاده از اجرای کد تزریق‌شده‌ی ناخواسته توسط مرورگر در صفحه، کاربر نهایی را در معرض خطر قرار داده و به آن حمله می‌کنند. به این ترتیب، مسیر دفاع در برابر حملات **XSS** در سمت کلاینت و هنگام خروجی دادن داده‌های ورودی کاربر که بالقوه خطرناک هستند، قرار دارد. دو روش اصلی برای جلوگیری از حملات **XSS** وجود دارد:

- **Filtering** – با فیلتر کردن یا پاکسازی داده‌های غیرقابل اعتمادی که از ورودی کاربر سرچشمه گرفته‌اند، نتیجه نهایی این است که داده‌ها اصلاح شده و متن اصلی موجود در آنها حذف می‌شود.

اگر به عنوان مثال، کاربری در یک وبلاگ بخواهد نظر دهد و مثالی از استفاده از برچسب‌های `<script>` ارائه دهد، فیلتر کردن بر اساس `allowlist/denylist`، هرگونه برچسب متخلف مانند `<script>` را حذف می‌کند، حتی اگر کاربر قصد اجرای مخرب این کد را در مرورگر نداشته باشد، بلکه فقط قصد چاپ آن و به اشتراک گذاشتن متن در وبسایت را داشته باشد.

مشکلات فیلترینگ این است که به پیکربندی‌های `allowlist/denylist` متکی است که می‌تواند در معرض تغییرات مکرر باشد، از این رو به نگهداری نیاز دارد و مستعد خطا است و معمولاً به منطق پیچیده دستکاری رشته نیاز دارد که اغلب مبتنی بر عبارات منظم است که به خودی خود می‌توانند به یک تهدید امنیتی تبدیل شوند یا به سادگی برای رسیدگی به تغییرات آینده و تغییرات رشته‌ای که برنامه‌نویس انتظار آن را نداشته است، به درستی نوشته نشده‌اند و بنابراین می‌توان از آنها عبور کرد.





• **Escaping/Encoding** - برخلاف فیلتر کردن، رمزگذاری داده‌های غیرقابل اعتماد، تمام ورودی‌هایی را که کاربر ارائه داده است، با فرار از کاراکترهای بالقوه مخرب با رمزگذاری کاراکتر نمایشی آنها، حفظ می‌کند. به عنوان مثال، اگر انتظار می‌رود ورودی از کاربر یک متن **HTML** باشد و همچنین به همین صورت با آن رفتار شود، در مواردی که ورودی `<script>alert('xss')</script>` باشد، می‌توان نماد `>` را به نمایش موجودیت **HTML** آن که `>` است، رمزگذاری کرد. این موجودیت کاراکتر همچنین دارای یک عدد مرتبط با آن است، بنابراین نماد `>` می‌تواند با رشته `60#&` نیز نمایش داده شود که منجر به همان رفتار رمزگذاری خواهد شد. مرورگرها می‌دانند که چگونه این موجودیت‌ها را تجزیه کرده و آنها را به درستی نمایش دهند.

نکته‌ی مهم در کدگذاری داده‌ها، کدگذاری آن‌ها با توجه به زمینه‌ی صحیح محل استفاده‌ی آن‌ها است. **HTML**، **JSON** و **CSS** از نظر کدگذاری با یکدیگر متفاوت هستند و یکی با دیگری مطابقت ندارد، بنابراین بسته به محل استفاده از ورودی، باید از شکل صحیح کدگذاری استفاده شود. به طور خلاصه، فیلتر کردن راه‌حل ایده‌آلی برای جلوگیری از حملات **XSS** نیست. اعتبارسنجی و فیلتر کردن داده‌ها باید روی ورودی داده‌های کاربر انجام شود و نه روی پردازش خروجی. از سوی دیگر، کدگذاری داده‌های خروجی، مسیر بهتری برای جلوگیری از حملات **XSS** است، زیرا هر داده‌ای را به صورت متن ساده ارائه می‌دهد که مرورگر فریب اجرای آن را نمی‌خورد.

📌 نکته : حملات **XSS** تکامل می‌یابند. مشخصات، مرورگرها و کل وب دائماً در حال تغییر هستند و فناوری‌های جدیدی را معرفی می‌کنند که برنامه‌های وب از آنها استفاده می‌کنند و امنیت نیز باید در مورد آنها اتخاذ شود. به همین ترتیب، حملات **XSS** دارای بردارهای حمله متنوعی برای سوءاستفاده هستند و دفاع در برابر آنها و وصله کردن آنها به طور فزاینده‌ای دشوارتر می‌شود.





Encoding libraries: node-esapi

OWASP پروژه **ESAPI** مخصوص به خود را دارد که هدف آن ارائه ابزارها، کتابخانه‌ها و **API** های مرتبط با امنیت است که توسعه‌دهندگان می‌توانند برای تأمین امنیت ضروری از آنها استفاده کنند. این پروژه به یک کتابخانه **Node.js** منتقل شده است که به عنوان یک بسته **npm** در دسترس است و **node-esapi** نام دارد. **node-esapi** را می‌توان مانند هر بسته **npm** دیگری نصب کرد و همچنین فایل **package.json** را با وابستگی‌های آن به‌روزرسانی کرد:

npm install node-esapi --save

پس از نصب، کتابخانه توابع کدگذاری را برای هر نوع داده‌ای که باید کدگذاری شود، ارائه می‌دهد، بنابراین دستورالعمل کلی زیر باید اعمال شود:

- وقتی قرار است داده‌های ورودی غیرقابل اعتماد در چارچوب یک کد جاوا اسکریپت قابل اجرا قرار گیرند، از کدگذاری جاوا اسکریپت استفاده کنید. برای مثال، انتظار می‌رود رشته‌ای از ورودی کاربر در یک کد منبع جاوا اسکریپت مانند `<script>showErrorMessage(userInput)</script>` استفاده شود.
- وقتی قرار است داده‌های ورودی غیرقابل اعتماد در نشانه‌گذاری **HTML** قرار گیرند، از کدگذاری **HTML** استفاده کنید. برای مثال، اگر قرار است داده‌ها در داخل تگ‌های `<div>`، `<span>` و غیره قرار گیرند.

برای **encode HTML**:

```
const esapi = require('node-esapi');
const esapiEncoder = esapi.encoder();

app.get('/', function (req, res, next) {
  // User input from query parameter - could contain malicious HTML/JavaScript
  // Example: <div><span>Example</span><script>alert('xss')</script></div>
  const userInputExample = req.query.userinput;

  // Encode the input so it's safe to output in HTML context
  const encodedInput = esapiEncoder.encodeForHTML(userInputExample);

  // Now encodedInput is safe to render in the HTML response
  // The malicious script tags will be displayed as text, not executed
});
```





نکته: کدگذاری برای سایر نمایش‌های داده. **node-esapi** همچنین شامل کدگذاری‌هایی برای **URL**، **CSS**، ویژگی‌های **HTML** و نمایش **Base64** داده‌ها است.

Encoding libraries: xss-filters

xss-filters یکی دیگر از کتابخانه‌های کدگذاری **XSS** از یاهو! است. این کتابخانه به گونه‌ای طراحی شده است که از مشخصات **HTMLD** برای پیاده‌سازی فیلترهای **XSS** پیروی کند و دائماً توسط محققان امنیتی یاهو! مورد بررسی قرار می‌گیرد. توجه به این نکته مهم است که **xss-filters** فقط برای استفاده در یک زمینه نشانه‌گذاری **HTML** در نظر گرفته شده است. شما نباید از آن برای ورودی‌های غیرقابل اعتماد کاربر در زمینه‌های دیگر مانند کد جاوا اسکریپت یا **CSS** یا سایر اشیاء خاص مانند برچسب‌های **<svg>** یا **<object>** یا **<embed>** استفاده کنید.

ياهو در جامعه **Node.js** کاملاً فعال است. آیا می‌دانستید که یاهو یک بازیگر فعال در جامعه **Node.js** است؟ آنها حدود صد بسته را به مخزن **npm** اضافه کرده‌اند، از جمله جاوا اسکریپت عمومی و کتابخانه‌های **frontend**، از جمله **Node.js**.

نصب فیلترهای **XSS**:

npm install xss-filters --save

استفاده از کتابخانه برای کدگذاری:

```
const xssFilters = require('xss-filters');

app.get('/', function (req, res, next) {
  // Get user input from URL query parameter
  const userInput = req.query.userinput;

  // Sanitize input for safe use inside HTML data context
  const safeUserInput = xssFilters.inHTMLData(userInput);

  // Now safeUserInput is safe to output in HTML context
  // The malicious content is neutralized
});
```





علاوه بر **API.inHTMLData** های دیگری نیز وجود دارند که برای مدیریت رمزگذاری داده‌های غیرقابل اعتماد در زمینه‌های دیگر وجود دارند:

- **HTML comments inHTMLComment** – برای رمزگذاری داده‌ها در نظرات **HTML** مانند

```
<!-- {{comment}} -->
```

- **HTML attributes** – برای رمزگذاری داده‌ها در ویژگی‌های **HTML**، لازم است از نماد نقل قول مناسب استفاده شده در زمینه ویژگی‌ها استفاده شود.

در مورد ویژگی‌های **HTML**، هنگام استفاده از نماد نقل قول تکی در ویژگی‌ها، از روش **inSingleQuotedAttr** استفاده کنید:

جاوا اسکریپت:

```
var safeUserInput = xssFilters.inSingleQuotedAttr(userInput);
```

:HTML

```
<input value= '{{safeUserInput}}' />
```

هنگام استفاده از علامت نقل قول دوتایی در ویژگی‌ها، از متد **inDoubleQuotedAttr** استفاده کنید:

جاوا اسکریپت:

```
var safeUserInput = xssFilters.inDoubleQuotedAttr(userInput);
```

:HTML

```
<input value= "{{safeUserInput}}" />
```





وقتی از هیچ نوع نقل قولی به عنوان ویژگی در عناصر **HTML** استفاده نمی‌کنید، برای مثال برای مشخص کردن کلمات کلیدی ویژگی **hidden** که روی یک عنصر **HTML** اعمال می‌شود و آن را نامرئی می‌کند، از متد **inUnQuote-dAttr** استفاده کنید: جاوااسکریپت:

جاوااسکریپت:

```
var safeUserInput = xssFilters.inUnQuotedAttr(userInput);
```

:HTML

```
<input name="csrfToken" value="{{csrfValue}}" {{safeUserInput}} />
```

برای تنظیم دقیق‌تر زمینه ورودی غیرقابل اعتماد از کاربر، مانند اینکه آیا از یک ورودی **URI** سرچشمه گرفته است یا خیر، می‌توان از یک روش خاص مانند زیر استفاده کرد:

```
var userURLInput = xssFilters.uriInHTMLData();
```

```
var userURIPathInput = xssFilters.uriPathInHTMLData();
```

```
var userURIFragmentInput = xssFilters.uriFragmentInHTMLData();
```

Summary

OWASP، اسکریپت‌نویسی بین‌سایتی (**XSS**) را در رتبه سوم از «۱۰ آسیب‌پذیری و بردار حمله برتر» قرار می‌دهد. به همین دلیل، آگاهی ما از نگرانی‌های امنیتی باید در مورد حملاتی که بسیار رایج و به راحتی قابل سوءاستفاده هستند، بالا باشد.

برای جلوگیری از آسیب‌پذیری‌های **XSS**، ما در مورد یکی از تکنیک‌های اساسی کاهش خطر، یعنی رمزگذاری یا **escape** کردن داده‌های خروجی، مطالبی آموختیم تا ورودی مخرب کاربر با تفسیر یک کد جاوا اسکریپت تزریق‌شده مخرب، مرورگر وب کاربر را به خطر نیندازد.

کتابخانه‌هایی که ما برای کاهش **XSS** بررسی کردیم، **node-esapi** خود **OWASP** و **xss-filters** یا **هستند**.

